



MASTER'S DEGREE IN AI

COMPUTER VISION – I

2025 - 2026

Universidade de Vigo



UNIVERSIDADE DA CORUÑA

PART 2: TEAM



Marta Núñez García

martanunez.garcia@usc.gal

Office hours (answer questions):
agree in advance by e-mail

3 theory sessions



Tiago Cortinhal

tiagofilipe.fernandes@usc.es



Carolina Pavez Higuera

carolina.pavez.higuera@usc.es

3 lab sessions

Only USC students



Centro Singular de Investigación
en **Tecnoloxías Intelixentes**

PART 2: TOPICS

4. Image segmentation

5. Feature descriptors

6. Matching

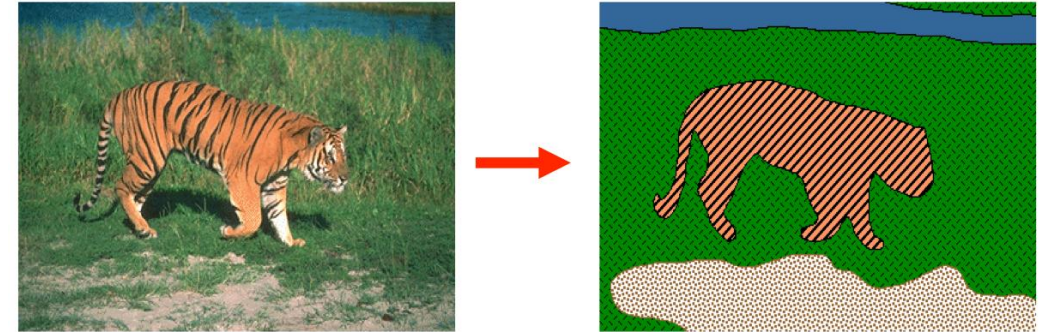


4. IMAGE SEGMENTATION

1. *Histogram-based segmentation*
2. *Clustering-based segmentation*
3. *Region growing and merging*
4. *Tagging*
5. *Morphological transformations*

IMAGE SEGMENTATION

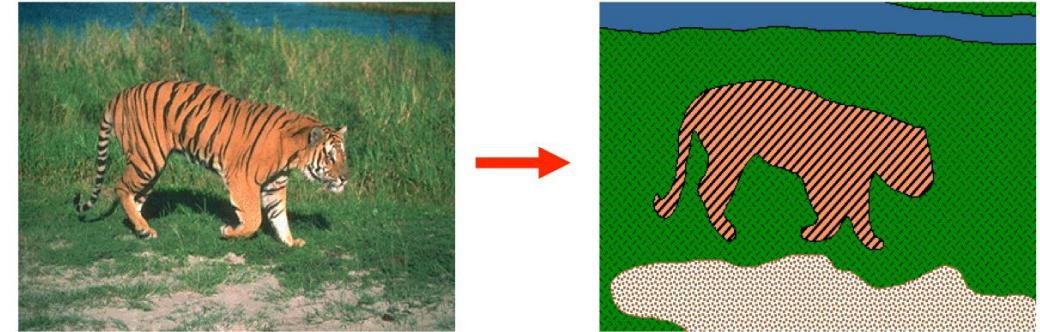
- **Image division** into separate regions / **segments**
- Split an image into non-overlapping regions based on some **homogeneity criteria** (intensity, color, texture,...)



AI Stanford <https://ai.stanford.edu/~syyeung/cvweb/tutorial3.html>

IMAGE SEGMENTATION

- **Image division** into separate regions / **segments**
- Split an image into non-overlapping regions based on some **homogeneity criteria** (intensity, color, texture,...)



AI Stanford <https://ai.stanford.edu/~syyeung/cvweb/tutorial3.html>

Each segment:

- Belongs to the **same object** / part of an object
- Has some **information attributes**
- Is made up of **connected pixels** / voxels (3D pixel)
- Can be identified by its **contour**



Segment Anything Model (SAM) – Meta AI, <https://segment-anything.com/>

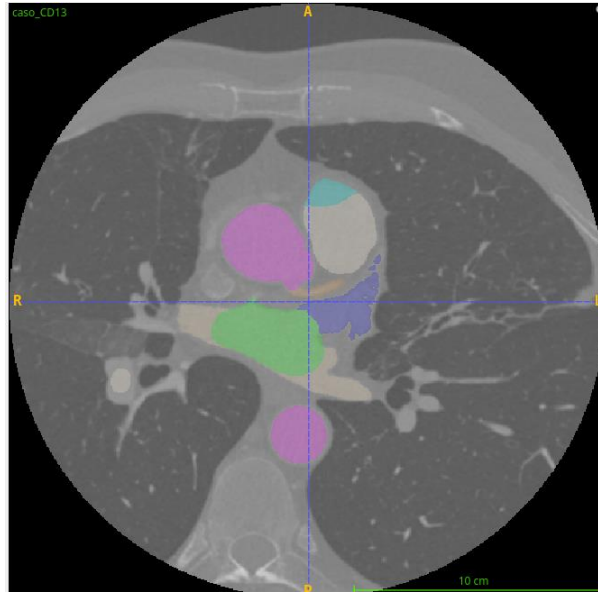
IMAGE SEGMENTATION

Many different approaches:

1. **Intensity**-based:
 - thresholding (histogram-based)
 - clustering
 2. Region growing
 3. **Contour** based, etc.
- 
- “Classical”
-
4. Multi-atlas segmentation (medical imaging)
 5. Deep Learning: U-net (+ 3D-Unet, V-net), nn-Unet, etc.
 6. Foundation models
- 
- ML / AI

INTENSITY-BASED THRESHOLDING

- Image segmentation based on **intensity difference** among pixels/voxels
- Works fine if objects to be segmented have **significantly different** intensities (color/gray level)
- Once the threshold is determined → transform different ranges of the image into integer numbers (**labels**)
- Key aspect: how to determine the **optimal threshold/s**?

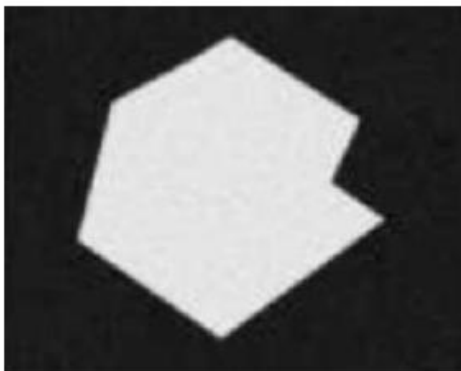


THRESHOLDING

- Algorithm:

$$g(x, y) = \begin{cases} 0, & f(x, y) < T \\ 1, & f(x, y) \geq T \end{cases}$$

Input



Output

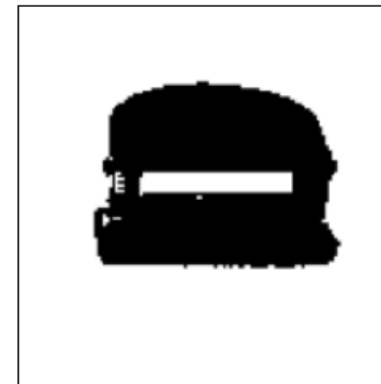


Binary mask /
segmentation

Input

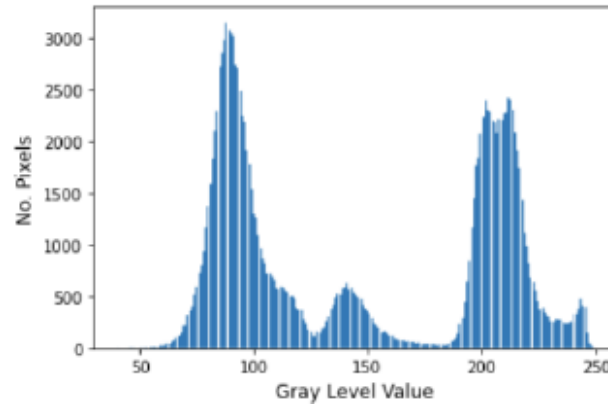


Output



OPTIMAL THRESHOLD?

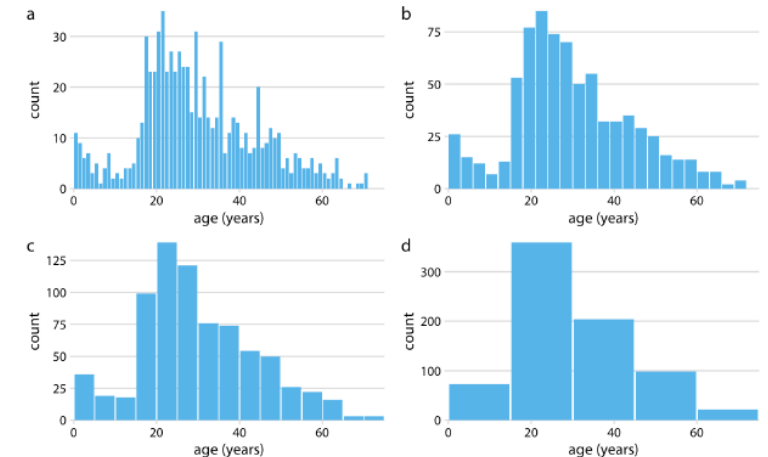
- Inspect the **histogram** of the image



- A few common distributions:

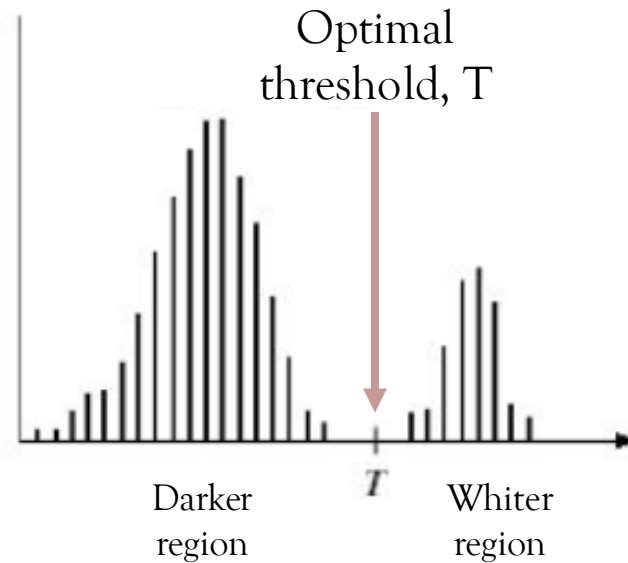


- Rectangles:
 - x = values (e.g. gray level)
 - y = counts (e.g. nb of pixels)
- Bin the data. The **width** of the bins (or alternatively, the nb of bins) is a crucial parameter



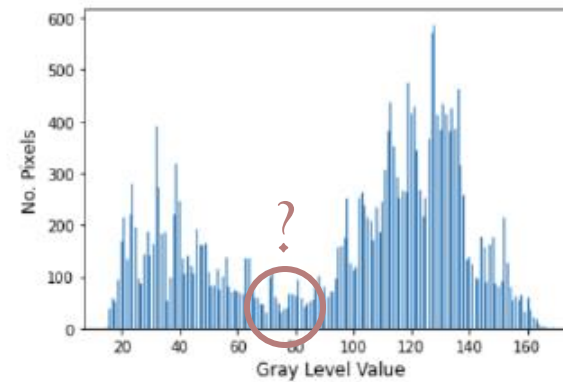
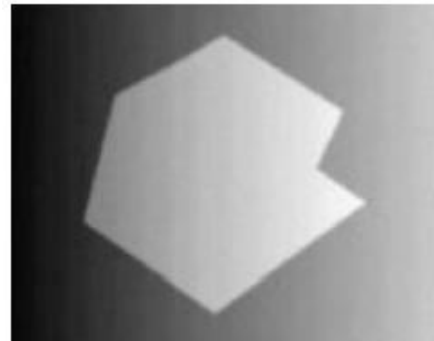
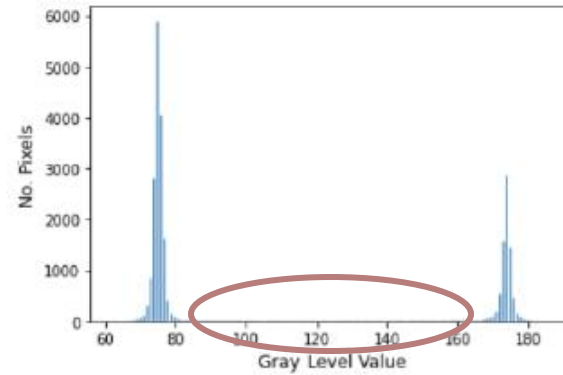
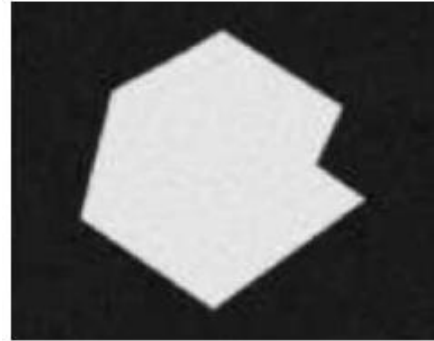
OPTIMAL THRESHOLD?

- **Bimodal** image:
 - Object surrounded by the **background** (BG)
 - The object becomes the **foreground** (FG)
 - BG and FG, to a greater or lesser extent, separated



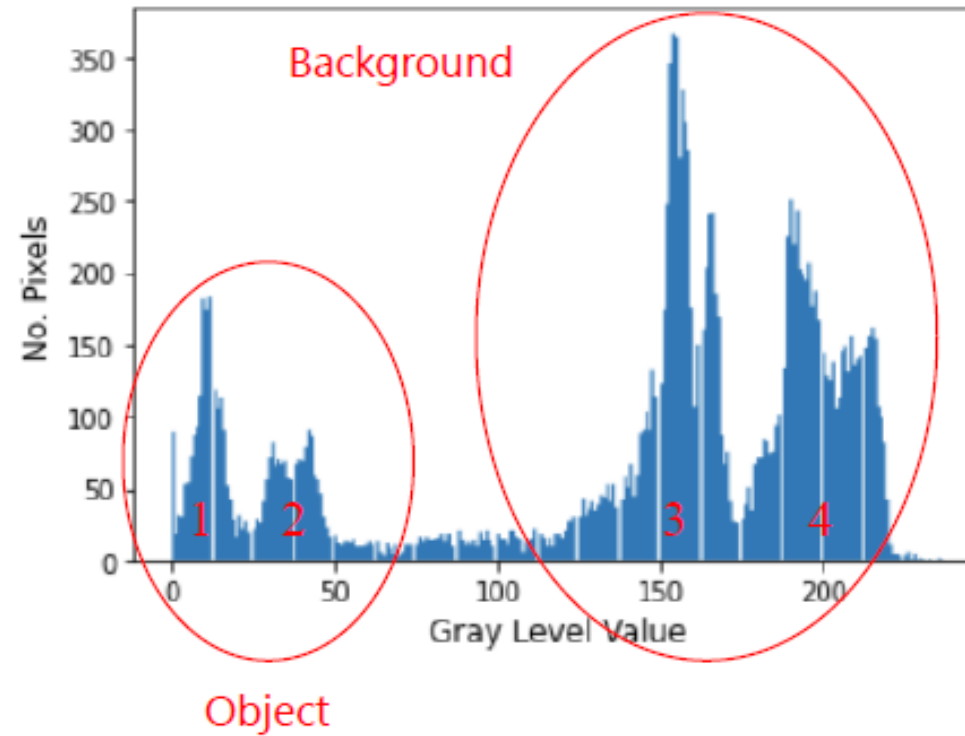
OPTIMAL THRESHOLD?

- **Clear** (e.g., wide range of suitable thresholds) vs. **unclear threshold**



OPTIMAL THRESHOLD?

- Each region could consist of 1 or more distributions (often Gaussians)



OPTIMAL THRESHOLD?: OTSU

- For **bimodal** histograms
- It searches for the best threshold, t , that **minimizes** the **intra-class variance**, $\sigma_{\omega}^2(t)$: weighted sum of the variances of the 2 classes

$$\sigma_{\omega}^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t)$$

Where the weights are the probabilities of the 2 classes:

$$\omega_0(t) = \sum_{i=0}^{t-1} p(i)$$

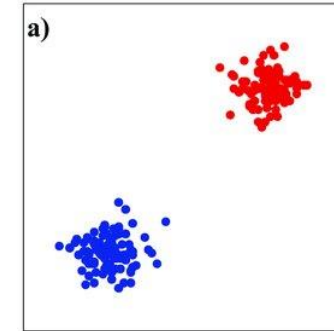
L : number of bins in the histogram
 $p(i)$: probability of pixel i (n_i/N)
 N : total number of pixels

$$\omega_1(t) = \sum_{i=t}^{L-1} p(i)$$

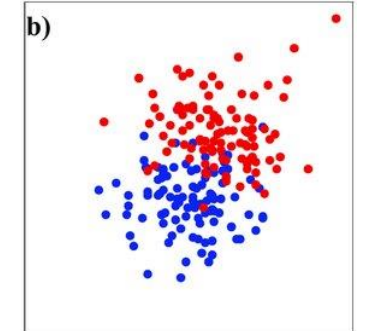
- If 2 classes, minimizing the intra-class variance is equivalent to **maximizing** the **inter-class variance**:

$$\begin{array}{ccccc} & \nearrow & \sigma^2 = \sigma_b^2(t) + & \sigma_{\omega}^2(t) & \\ \text{Total} & & \text{Inter-class} & & \text{Intra-class} \\ \text{variance} & & \text{variance} & & \text{variance} \end{array}$$

Low intra-class variance
High inter-class variance



High intra-class variance
Low inter-class variance

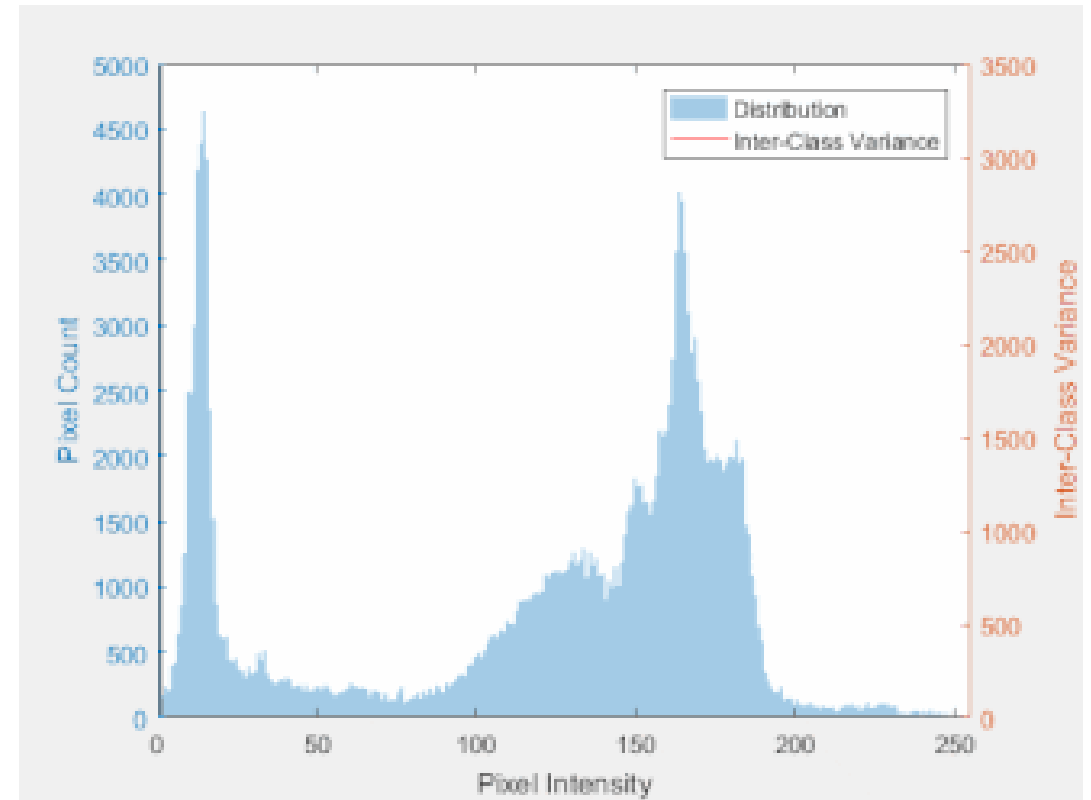
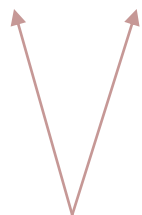


Jânio Coutinho Canuto. *Biomechanical online signature modeling applied to verification*. 2014.

OTSU ALGORITHM

1. Compute histogram and probabilities of each intensity level
2. Set initial $t=0$, compute $\sigma_b^2(0)$
3. For $t=1 \dots \text{max intensity}$:
 - Compute $\sigma_b^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$
4. Optimal threshold: $\max\{\sigma_b^2(t)\}$

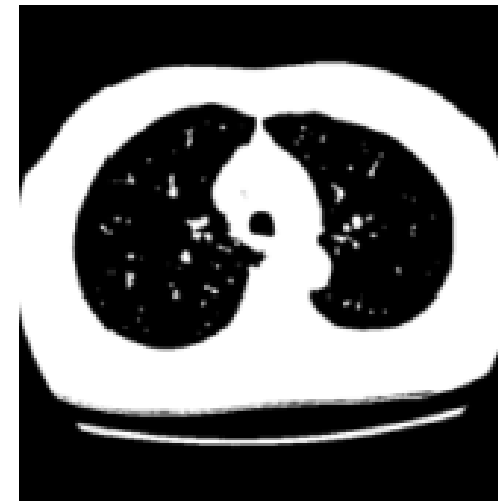
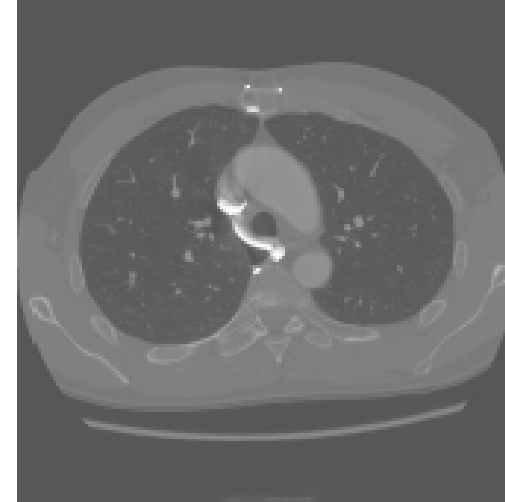
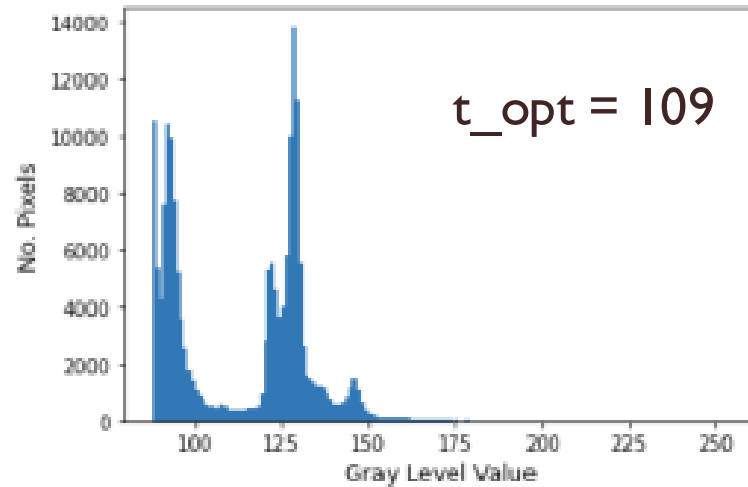
Class means



Wikipedia: Otsu's method

OTSU ALGORITHM: EXAMPLE

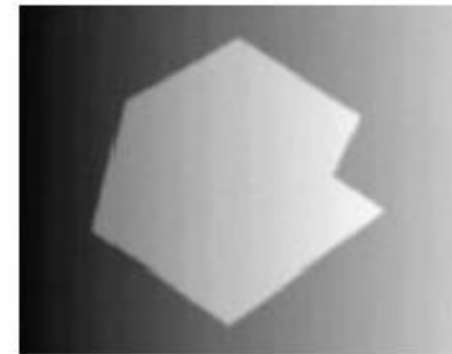
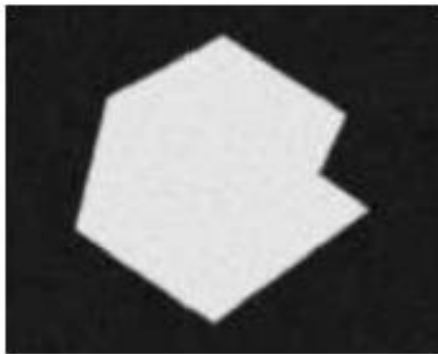
- Python code → Lab tomorrow



ADAPTIVE THRESHOLDING

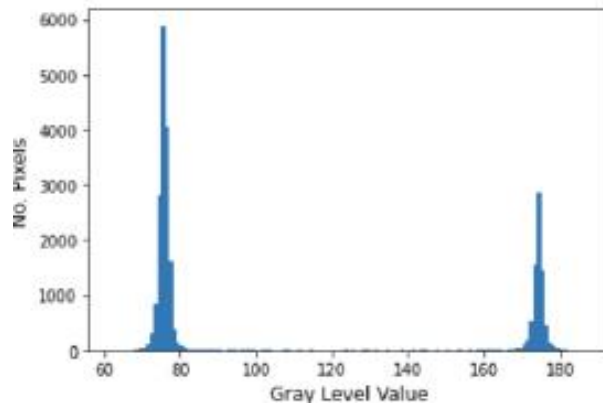
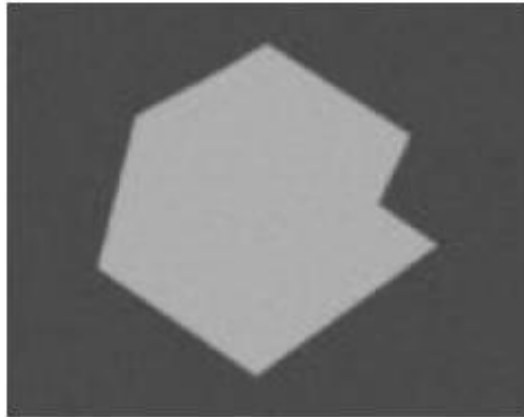
- **Illumination** transforms homogeneous images into non-homogeneous ones
- This effect can be modeled as the **multiplication** of the input image by a matrix representing the inhomogeneity due to illumination

$$f(x, y) = i(x, y) r(x, y)$$

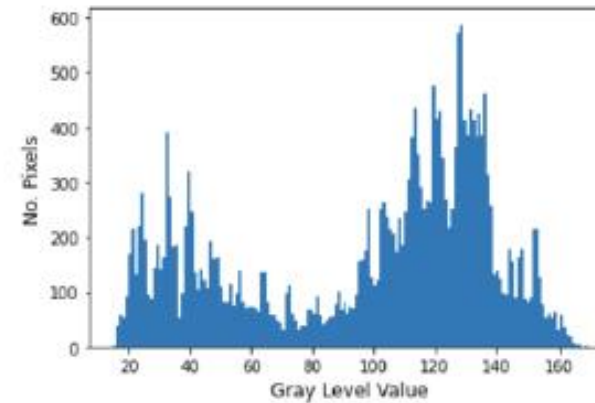
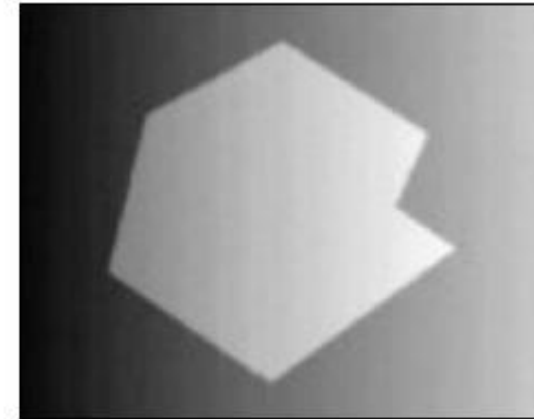


ADAPTIVE THRESHOLDING

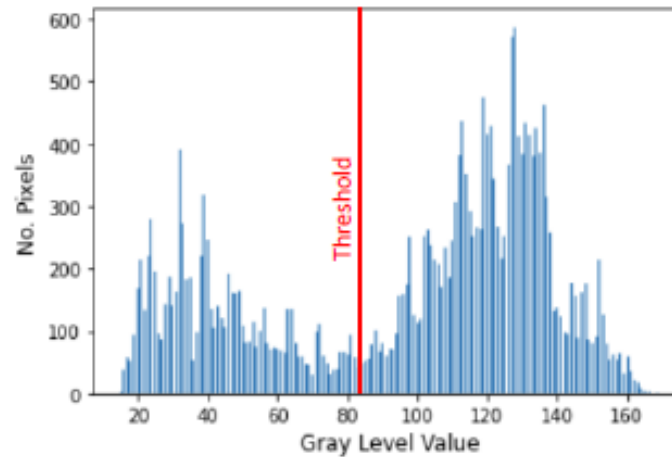
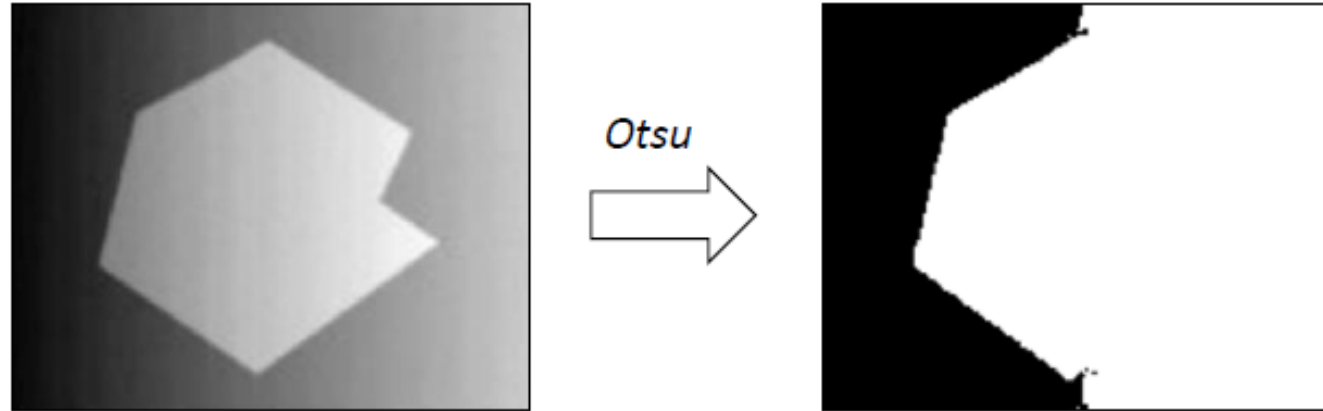
Homogeneous image



Non-homogeneous image

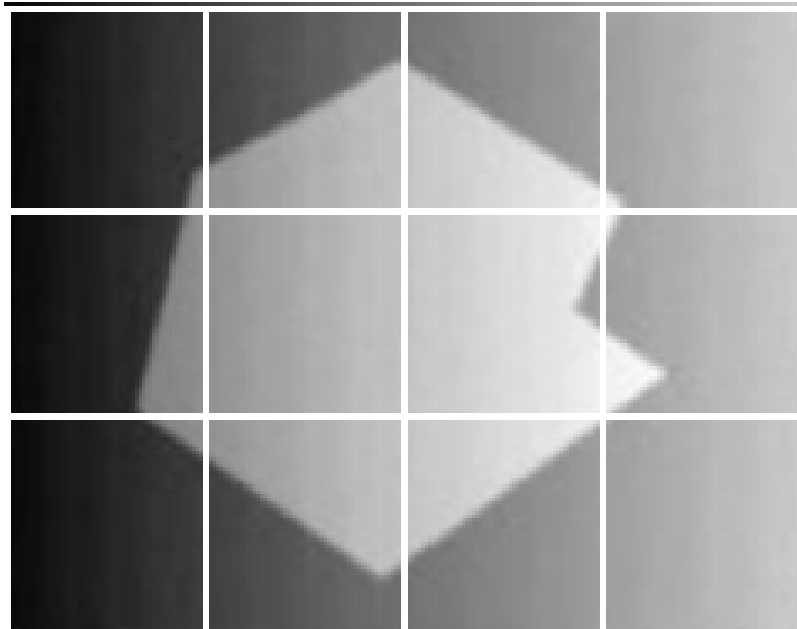


ADAPTIVE THRESHOLDING

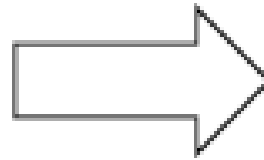


ADAPTIVE THRESHOLDING

- Different thresholds are computed for **smaller** regions in the image
- New parameter, w , **window size** used to spatially divide the image

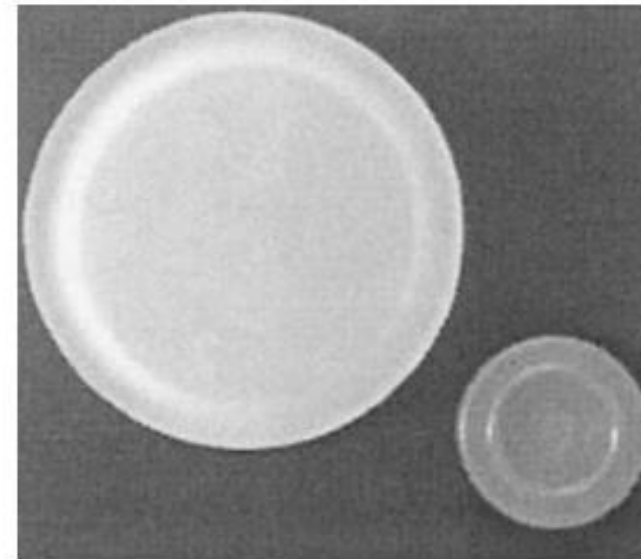
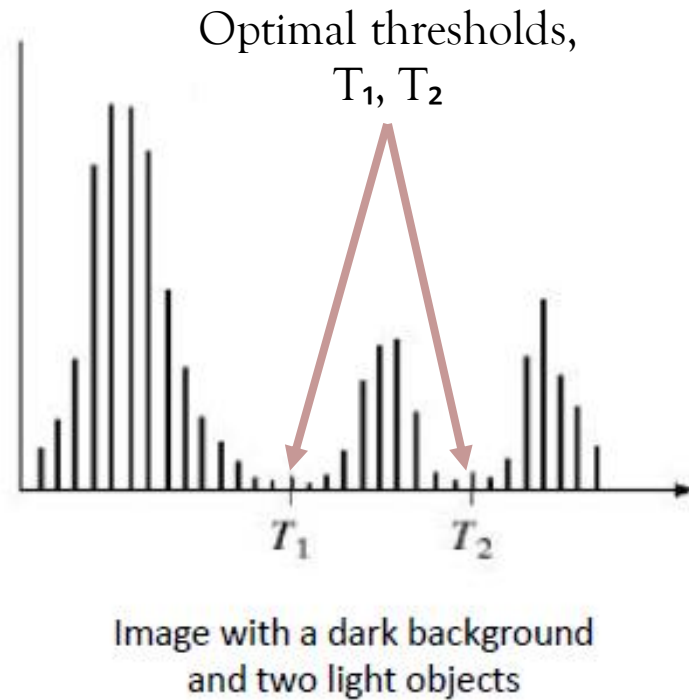


Adaptive



MULTILEVEL THRESHOLDING

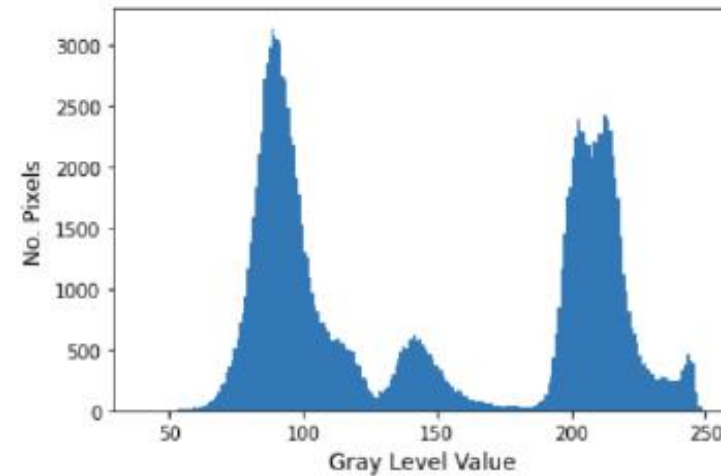
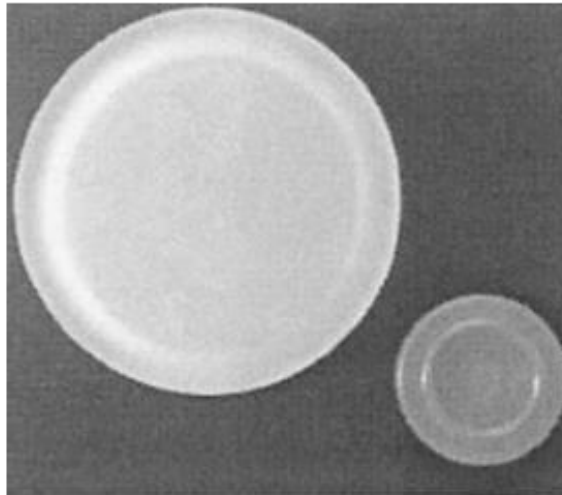
- **Multimodal** image: **several** objects surrounded by the background



MULTILEVEL THRESHOLDING

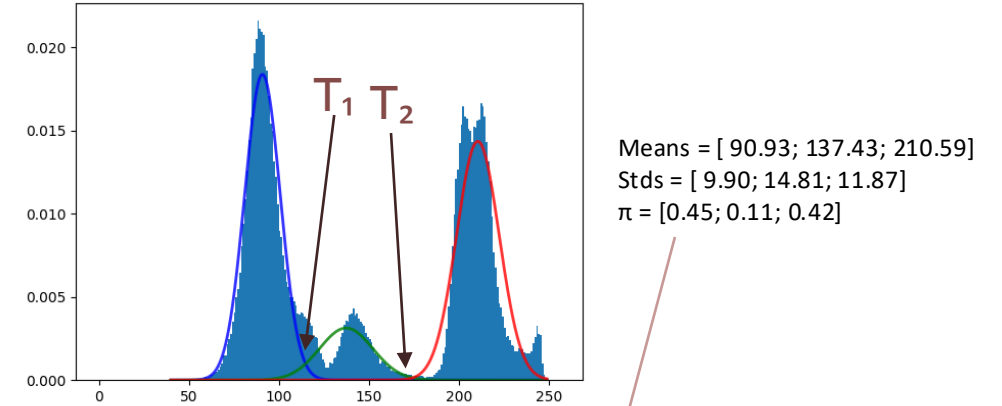
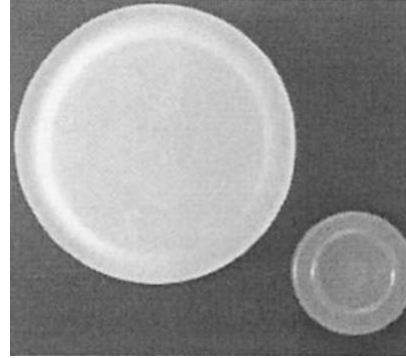
- **Several** optimal thresholds

$$g(x, y) = \begin{cases} V_0, & f(x, y) < T_0 \\ V_1, & T_0 \leq f(x, y) \leq T_1 \\ V_2, & T_1 \leq f(x, y) \leq T_2 \\ \dots & \dots \\ V_n, & f(x, y) > T_m \end{cases}$$



GAUSSIAN MIXTURE MODEL

- Bimodal or multimodal distributions
- Parametric: **number** of Gaussians
- Expectation maximization
- Use estimated parameters (mean, std, weight) to compute optimal thresholds



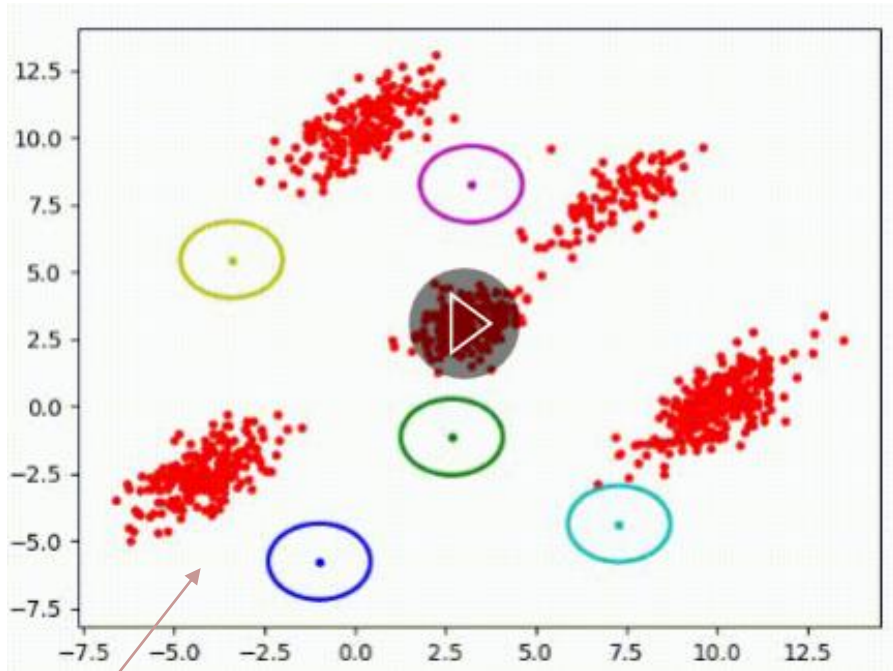
weight π , ($\pi_1 + \pi_2 + \pi_3 = 1$) to account for the fact that we do not have an equal number of samples from each category

Algorithm

- Decide the **nb of clusters** (domain knowledge)
- **Initiate** mean, covariance and weight parameters per cluster (often **randomly**)
- **Expectation Maximization:**
 - Expectation Step (E step): Calculate the probability of each data point belonging to each distribution, then evaluate the likelihood function using the current estimate for the parameters
 - Maximization step (M step): **Update** the previous mean, covariance, and weight parameters to **maximize** the expected likelihood found in the E step
 - Repeat these steps until the model converges

Likelihood function: statistical function that measures the probability of observing a set of data given a set of parameters in a statistical model

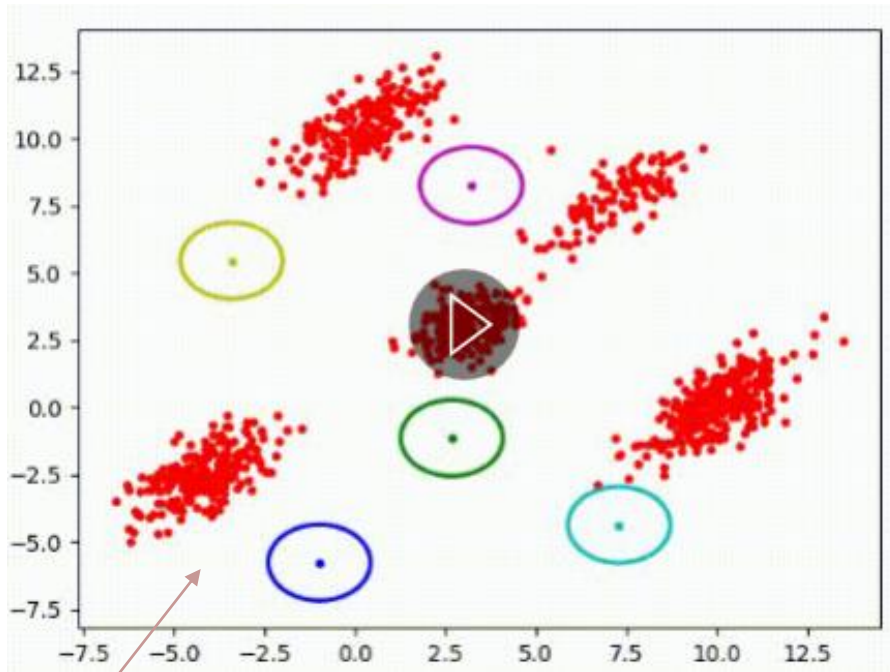
GAUSSIAN MIXTURE MODEL



Random initialization
of mean, std & weight

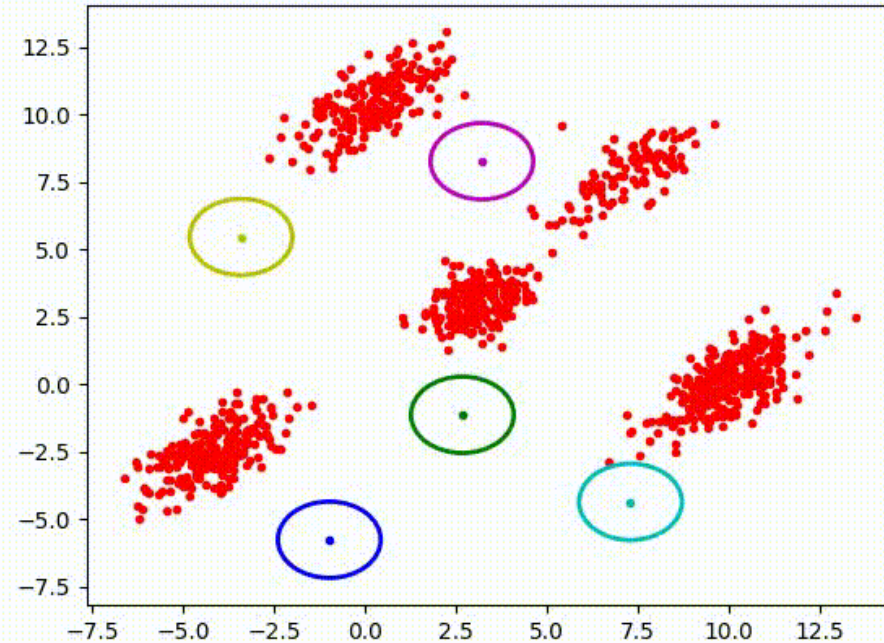
<https://mlpack.org/gsocblog/Sangyeon2019WeekTen.html>

GAUSSIAN MIXTURE MODEL



Random initialization
of mean, std & weight

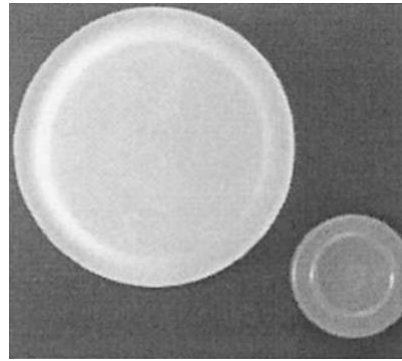
Iterative update using the EM algorithm



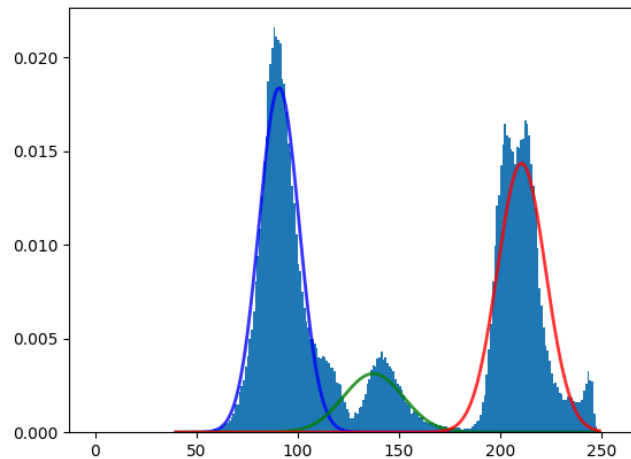
<https://mlpack.org/gsocblog/Sangyeon2019WeekTen.html>

GAUSSIAN MIXTURE MODEL

- Parametric: number of Gaussians

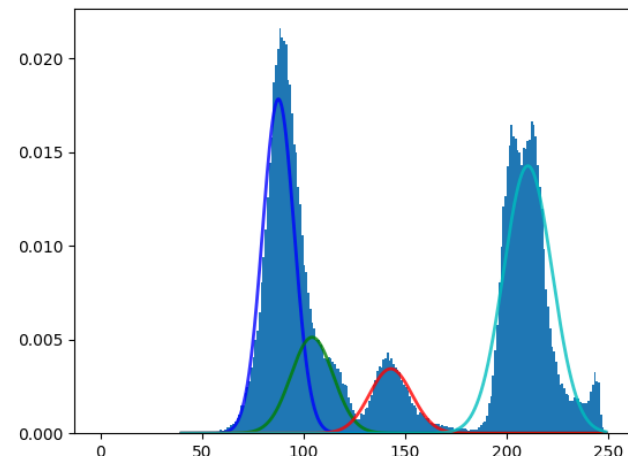


$n = 3$



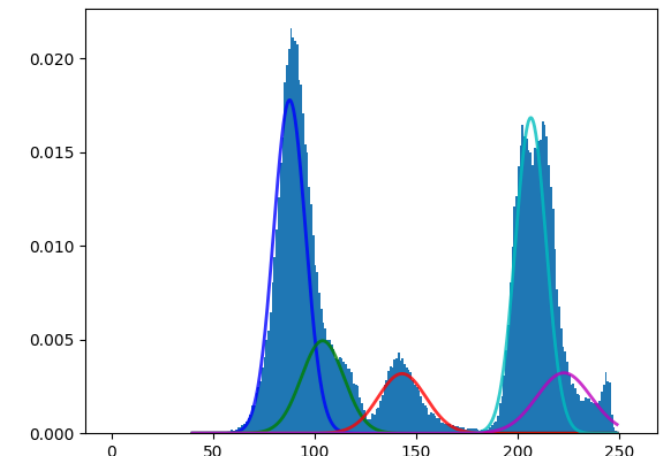
Means = [90.93; 137.43; 210.59]
Stds = [9.90; 14.81; 11.87]
 π = [0.45; 0.11; 0.42]

$n = 4$



Means = [87.71; 104.30; 143.01; 210.53]
Stds = [7.79; 10.39; 10.41; 11.96]
 π = [0.34; 0.13; 0.089; 0.42]

$n = 5$



Means = [87.82; 104.08; 143.16; 206.58; 222.87]
Stds = [7.86; 10.47; 11.75; 7.53; 13.33]
 π = [0.35; 0.12; 0.093; 0.31; 0.10]

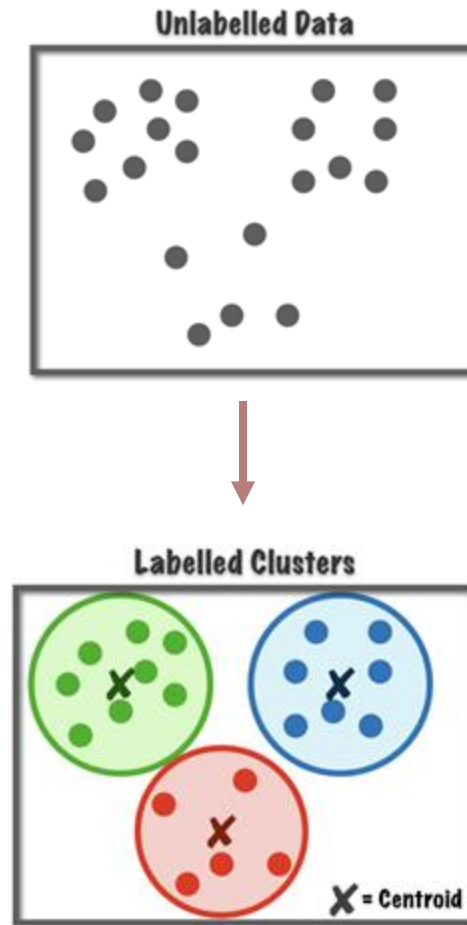
IMAGE SEGMENTATION

Many different approaches:

1. **Intensity**-based:
 - thresholding (histogram-based)
 - clustering
 2. Region growing
 3. **Contour** based, etc.
- 
- “Classical”
-
4. Multi-atlas segmentation (medical imaging)
 5. Deep Learning: U-net (+ 3D-Unet, V-net), nn-Unet, etc.
 6. Foundation models
- 
- ML / AI

CLUSTERING

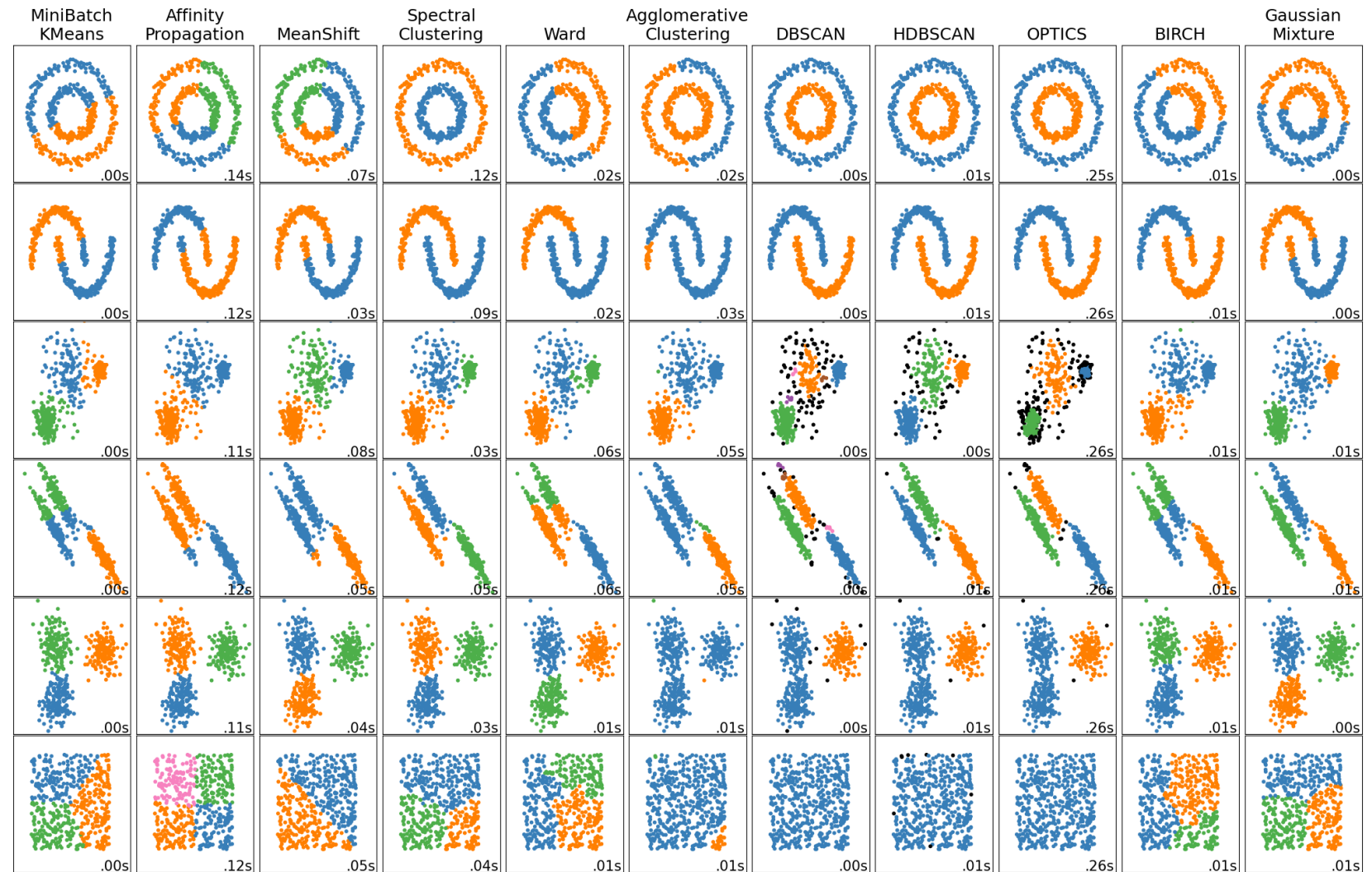
- **Group** together pixels with **similar** image **characteristics**
- Represent them as a single entity (**cluster**)
- Image characteristics could be:
 1. Gray levels: $p = (x, y, I(x, y)) \in \mathbb{R}^3$
 2. Colors: $p = (x, y, R(x, y), G(x, y), B(x, y)) \in \mathbb{R}^5$
 3. Textures: $p = (x, y, \text{vector_local_features}) \in \mathbb{R}^n$
 4. Etc.
- Another way of performing image **segmentation** but **guided by the data itself** (i.e., no need to estimate thresholds)



<https://towardsdatascience.com/k-means-a-complete-introduction-1702af9cd8c>

CLUSTERING

- There are many methods
- None is better than another
- It depends on the use case
- Domain knowledge is important



<https://scikit-learn.org/stable/modules/clustering.html>

CLUSTERING

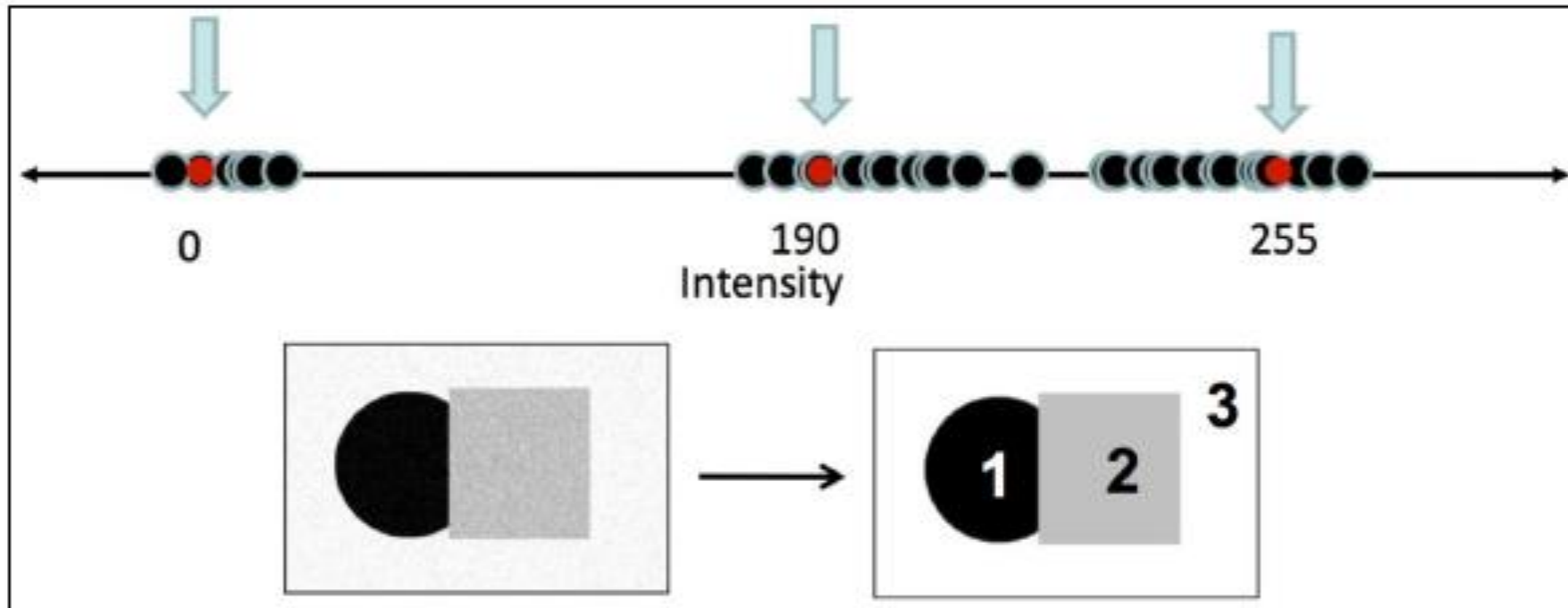
- There are many methods
- None is better than another
- It depends on the use case
- Domain knowledge is important

Method name	Parameters	Scalability	Usecase	Geometry (metric used)
K-Means	number of clusters	Very large <code>n_samples</code> , medium <code>n_clusters</code> with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters, inductive	Distances between points
Affinity propagation	damping, sample preference	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry, inductive	Graph distance (e.g. nearest-neighbor graph)
Mean-shift	bandwidth	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry, inductive	Distances between points
Spectral clustering	number of clusters	Medium <code>n_samples</code> , small <code>n_clusters</code>	Few clusters, even cluster size, non-flat geometry, transductive	Graph distance (e.g. nearest-neighbor graph)
Ward hierarchical clustering	number of clusters or distance threshold	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, transductive	Distances between points
Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, non Euclidean distances, transductive	Any pairwise distance
DBSCAN	neighborhood size	Very large <code>n_samples</code> , medium <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, outlier removal, transductive	Distances between nearest points
HDBSCAN	minimum cluster membership, minimum point neighbors	large <code>n_samples</code> , medium <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, outlier removal, transductive, hierarchical, variable cluster density	Distances between nearest points
OPTICS	minimum cluster membership	Very large <code>n_samples</code> , large <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, variable cluster density, outlier removal, transductive	Distances between points
Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation, inductive	Mahalanobis distances to centers
BIRCH	branching factor, threshold, optional global clusterer.	Large <code>n_clusters</code> and <code>n_samples</code>	Large dataset, outlier removal, data reduction, inductive	Euclidean distance between points
Bisecting K-Means	number of clusters	Very large <code>n_samples</code> , medium <code>n_clusters</code>	General-purpose, even cluster size, flat geometry, no empty clusters, inductive, hierarchical	Distances between points

<https://scikit-learn.org/stable/modules/clustering.html>

CLUSTERING

- Some methods involve choosing **cluster centers: representatives** of the different pixel intensities



AI Stanford lab

CLUSTERING: K-MEANS

- **Objective:** partition / divide the observations into **k** sets, $S = \{S_1, S_2, \dots, S_k\}$

- **Iterative algorithm:**

1. Initialize **k** clusters given a **random** initial set of **means (cluster centers)**: $m_1^{(0)}, \dots, m_k^{(0)}$
2. Assign each pixel to the **closest center**

$$S_i^{(t)} = \left\{ x_p : \|x_p - m_i^{(t)}\|^2 \leq \|x_p - m_j^{(t)}\|^2 \quad \forall j, 1 \leq j \leq k \right\}$$

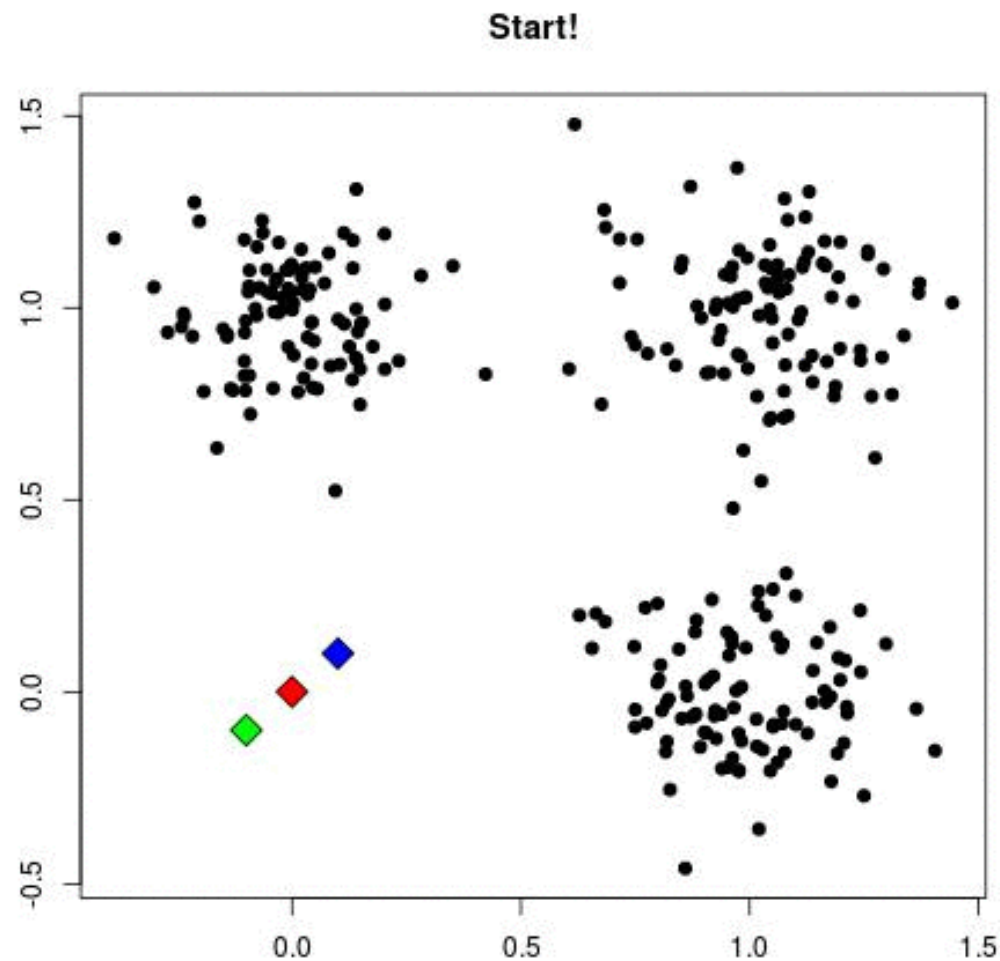
where each x_p is assigned to only one $S^{(t)}$ even if it could be assigned to more (equal distance)

3. **Update** cluster centers = average of the current pixels in the cluster

$$m_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{x_j \in S_i^{(t)}} x_j$$

4. Repeat steps 2 and 3 until convergence (no change in centers)

CLUSTERING: K-MEANS



<https://mubaris.com/posts/kmeans-clustering/>

CLUSTERING: K-MEANS

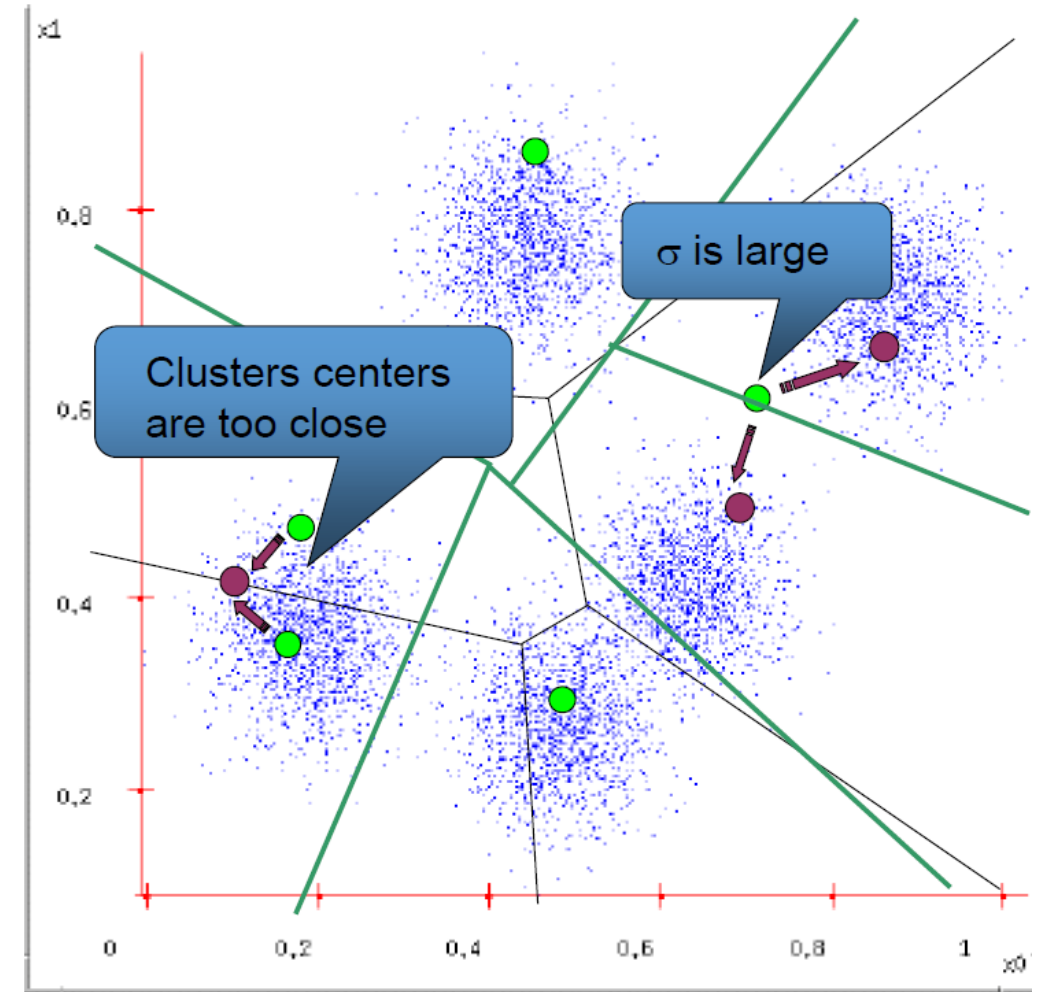
- **Splitting** when large values of variance:

$$\sigma_i = \sum_{j=1}^{N_i} (x_{ij} - M_i)^2 \quad \text{with} \quad M_i = \frac{1}{N_i} \sum_{j=1}^{N_i} X_{ij}$$

- **Merging** when centers too close

$$Dist(M_k, M_l) = \sqrt{\sum_{p=1}^m (M_{kp} - M_{lp})^2}$$

- If one cluster is split, 2 clusters must be merged (fixed number of clusters in k-means)



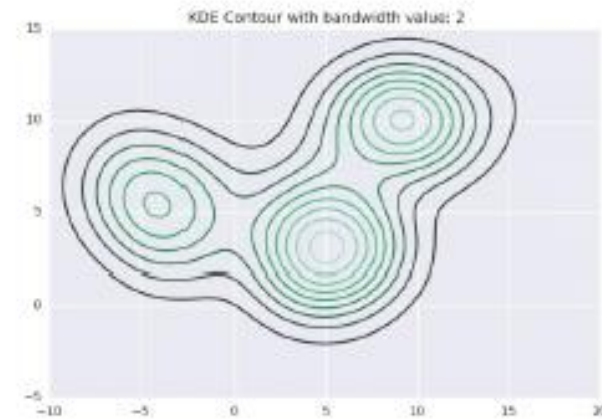
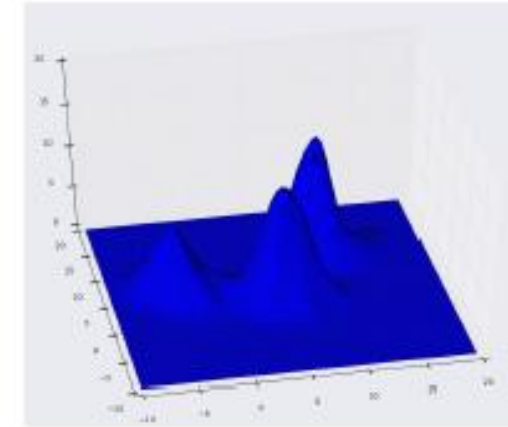
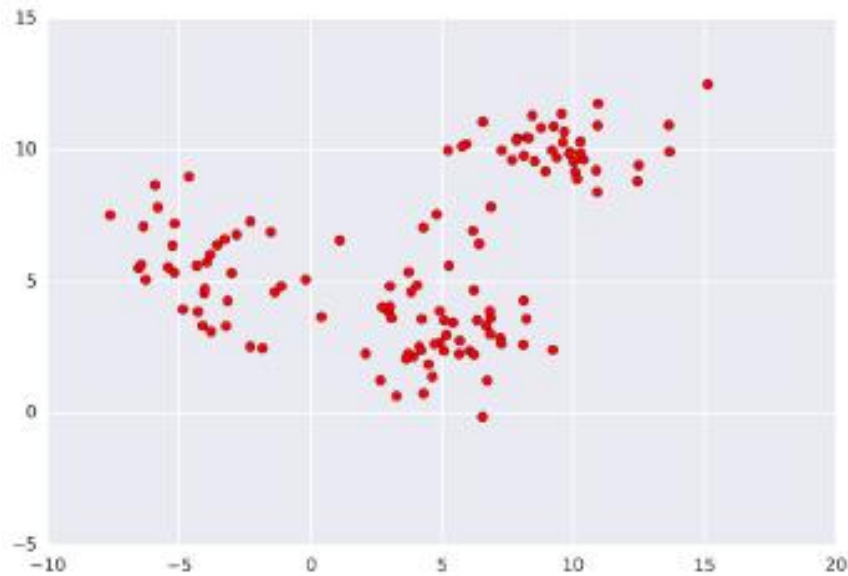
CLUSTERING: K-MEANS

- Python code → Lab tomorrow



CLUSTERING: MEAN-SHIFT

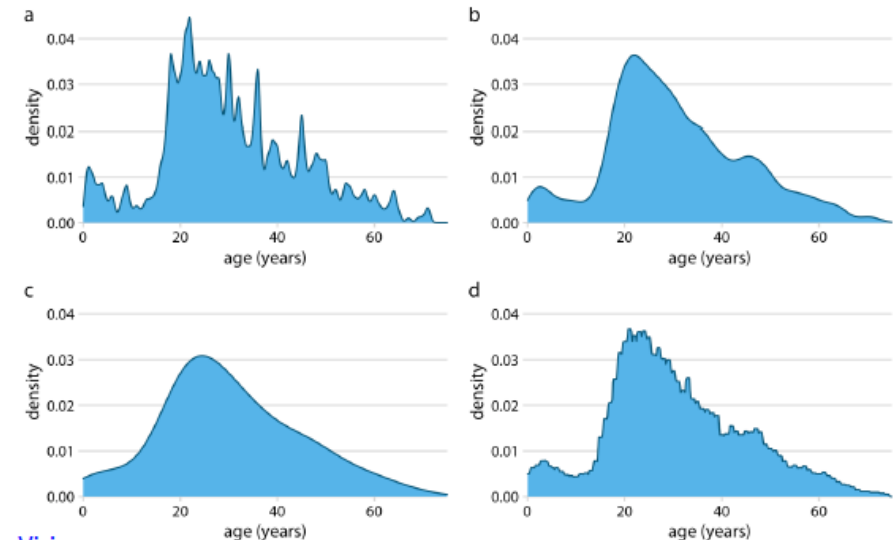
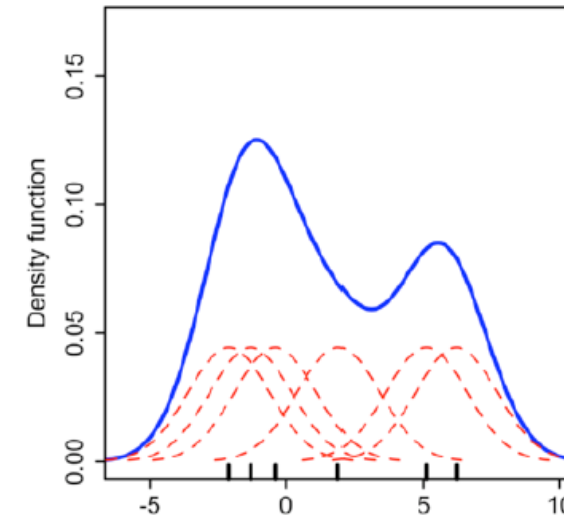
- Based on Kernel Density Estimation (KDE)
- Different ways of representing density:



- Mean shift creates a KDE for the dataset

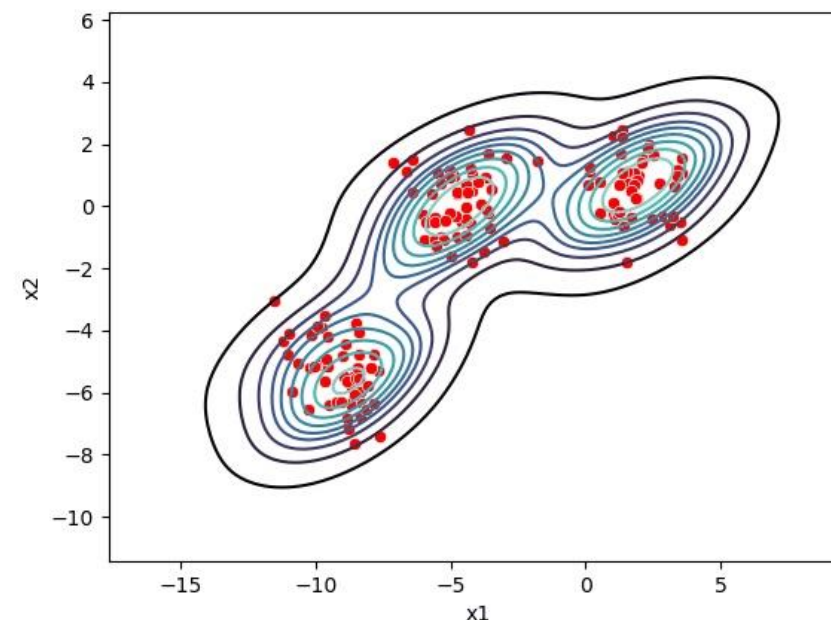
CLUSTERING: MEAN-SHIFT (KDE)

- A **density plot** visualizes the underlying **probability distribution** of the data
- Continuous curve
- Data scaled such that the area under the curve equals 1
- Estimated from the **data** → **kernel density estimation** method:
 - Continuous curves (the **kernel**) are drawn at the location of each data point
 - Most widely used kernel: Gaussian kernel
 - All curves are **added up**
 - Relevance of the **bandwidth** parameter (similar to width of bins in histograms)



CLUSTERING: MEAN-SHIFT

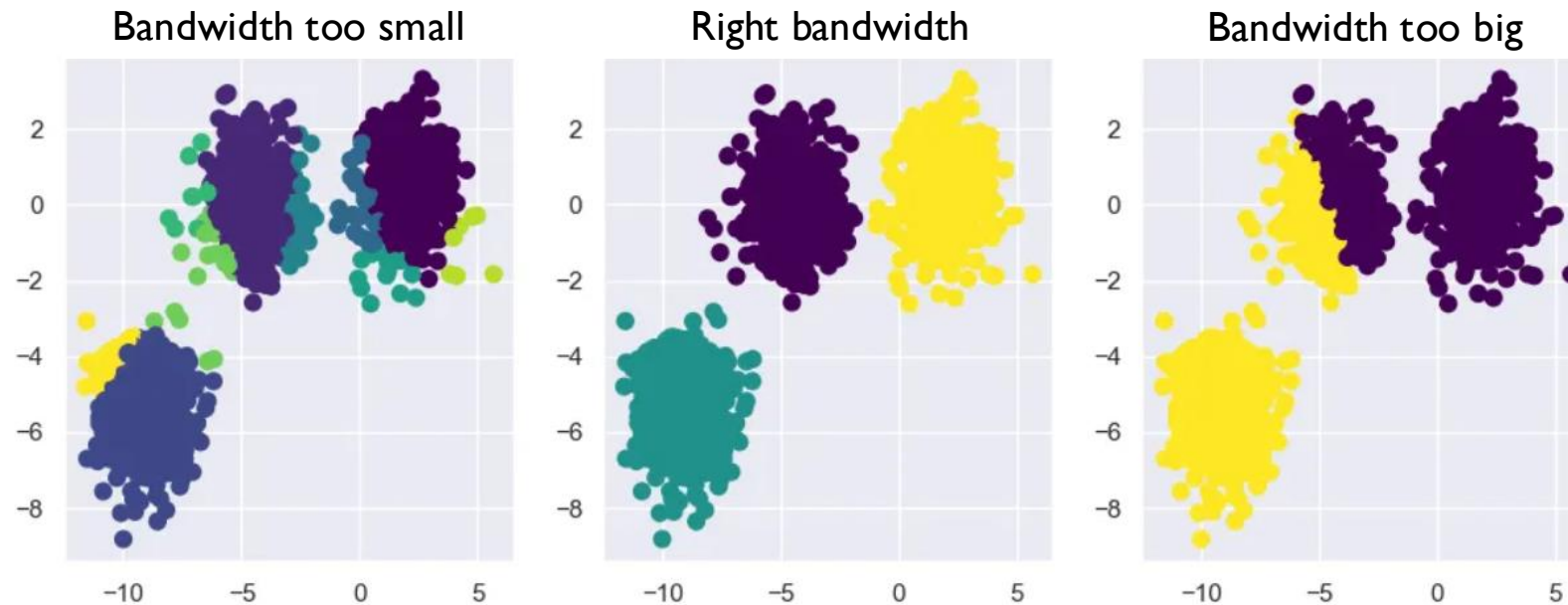
- Algorithm:
 1. Create a sliding window for each data-point
 2. Each windows is **shifted** towards **higher density regions**: shift centroid (center of the sliding window) to the data-points' mean within the sliding window. Iterate until no shift yields a higher density (number of points in the sliding window)
 3. **Delete overlapping windows** (keep only the window containing the most points)
 4. Assign data points to the sliding window in which they reside
- No need to choose the number of clusters in advance
- Not **dependent on the initialization**: at the start, each point is a cluster



<https://ml-explained.com/blog/mean-shift-explained>

CLUSTERING: MEAN-SHIFT

- Influence of the bandwidth parameter



- Extreme cases:
 - Extremely small $\rightarrow$ each point is a cluster
 - Extremely big $\rightarrow$ only 1 cluster containing all data points

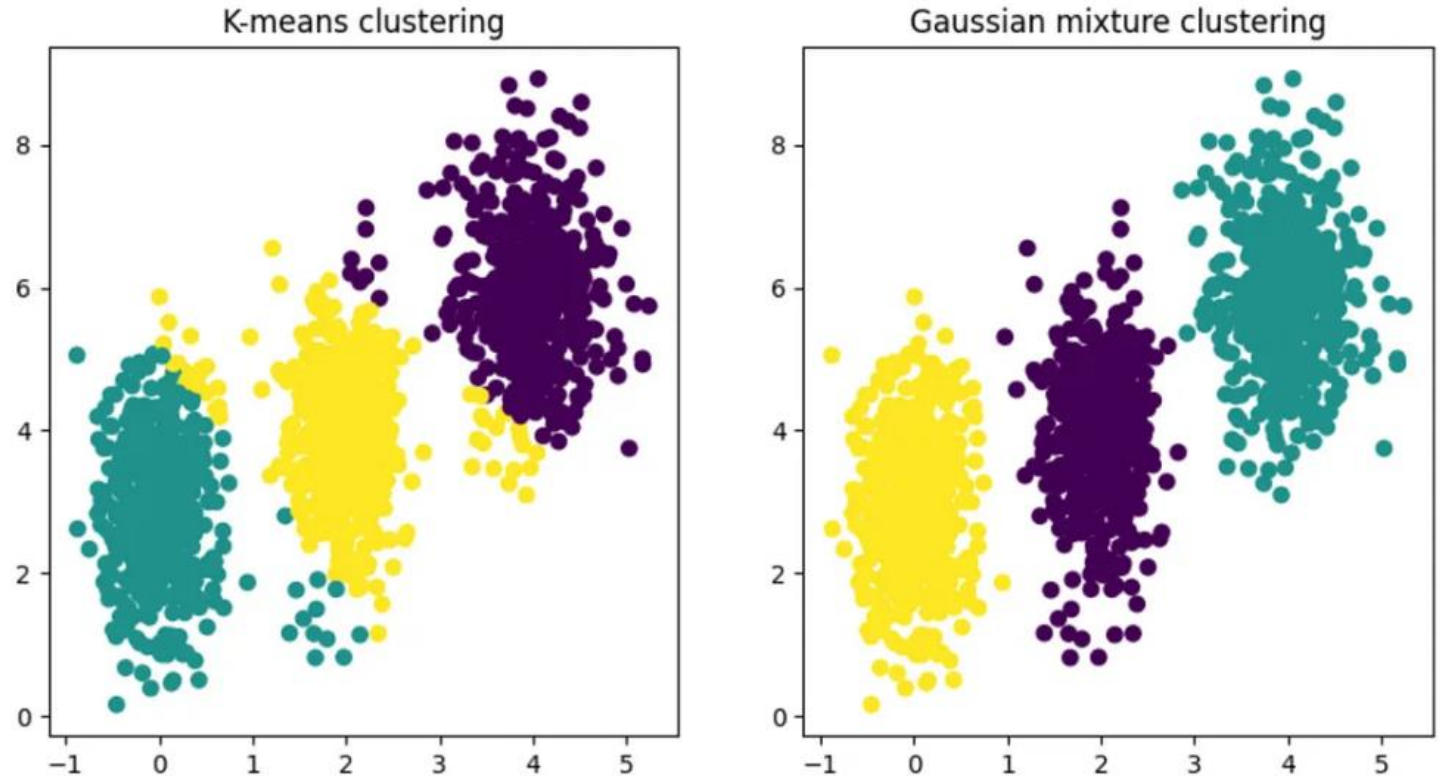
CLUSTERING: MEAN-SHIFT

- Python code → Lab tomorrow



CLUSTERING WITH GMM

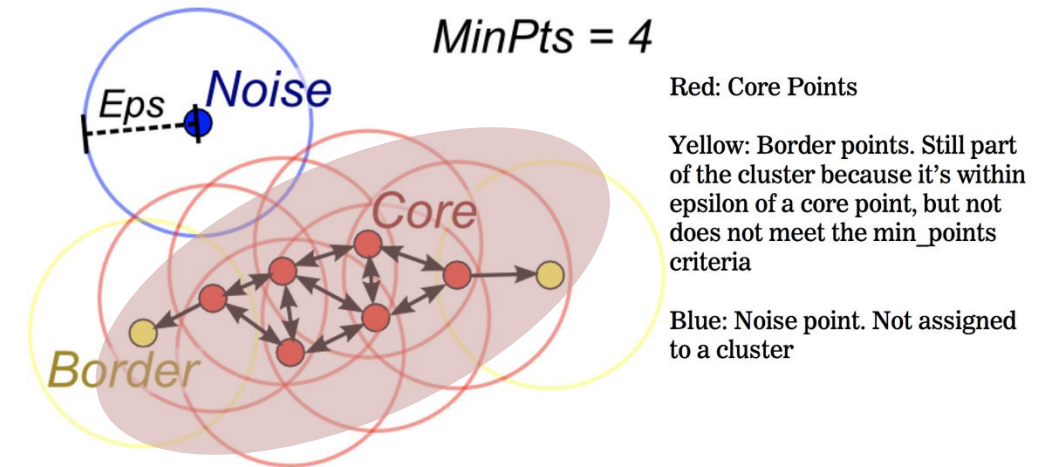
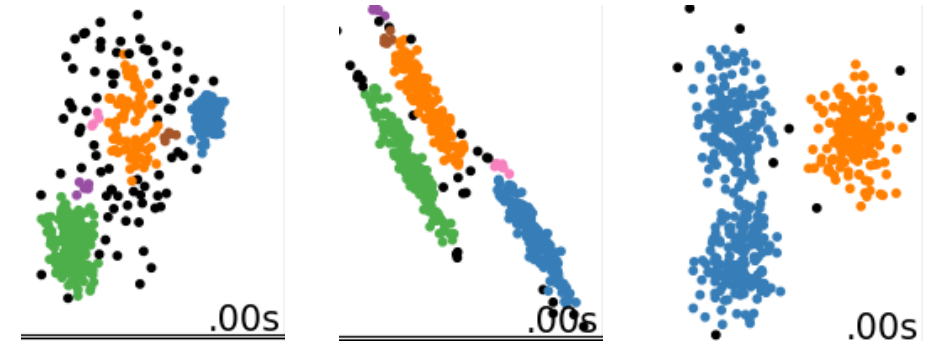
- GMM represents clusters using **Gaussian distributions**
- It does not only capture the means (e.g., k-means) of the clusters but also the **covariances**
- It allows for modeling ellipsoidal shapes



<https://towardsdev.com/clustering-with-gaussian-mixture-models-c2c3ecdc6640>

CLUSTERING: DBSCAN

- **Density-based** spatial clustering of applications with noise (DBSCAN)
- Groups together points that are **closely packed together** (points with many nearby neighbors)
- Mark as **outliers** points that lie alone in low-density regions
- ϵ : radius of a neighborhood
- Points classified as:
 - **Core points**: at least minPts points are within distance ϵ of it (including p)
 - (Directly-) **reachable points**: within distance ϵ from core point
 - **Outliers**: points not reachable from any other point are outliers or noise points
- 2 parameters: ϵ and minPts (no nb of clusters in this case)



<https://elutins.medium.com/dbscan-what-is-it-when-to-use-it-how-to-use-it-8bd506293818>

CLUSTERING: DENSITY-BASED

- Cluster **non-connected** image regions
- E.g., fat segmentation noisy CT image:

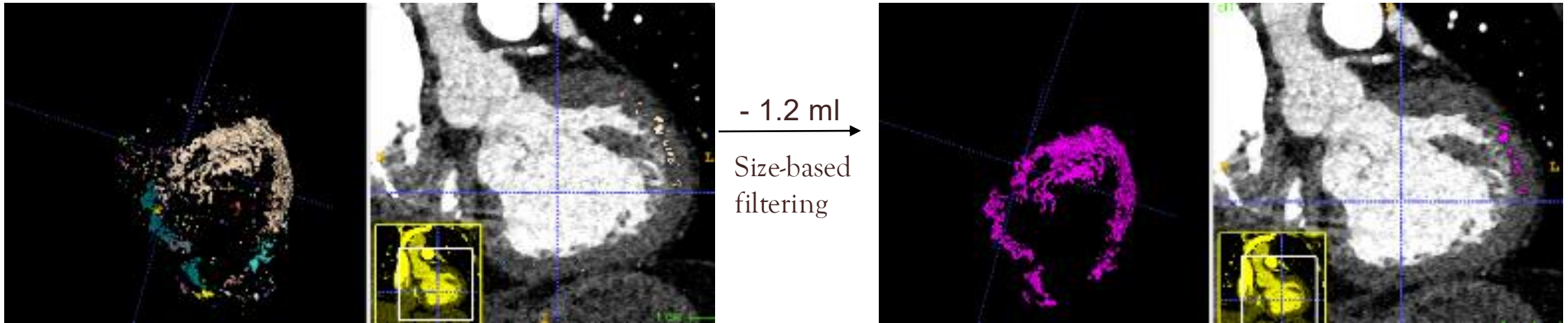


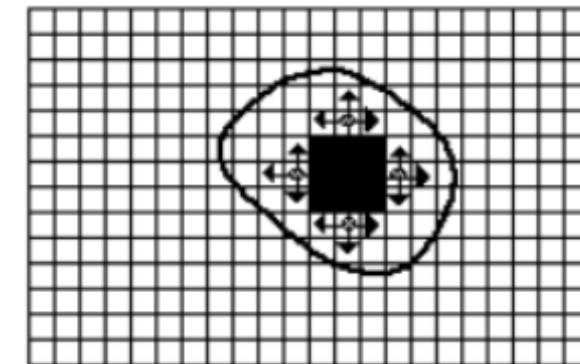
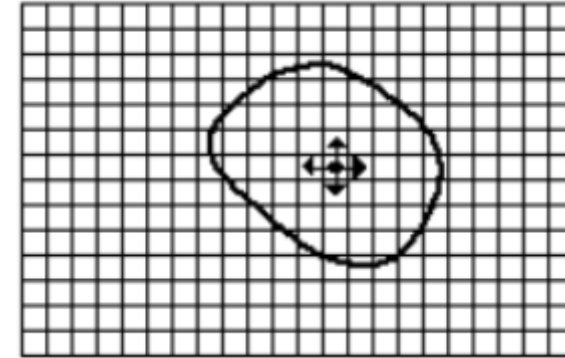
IMAGE SEGMENTATION

Many different approaches:

1. **Intensity**-based:
 - thresholding (histogram-based)
 - clustering
 2. Region growing
 3. **Contour** based, etc.
- 
- “Classical”
-
4. Multi-atlas segmentation (medical imaging)
 5. Deep Learning: U-net (+ 3D-Unet, V-net), nn-Unet, etc.
 6. Foundation models
- 
- ML / AI

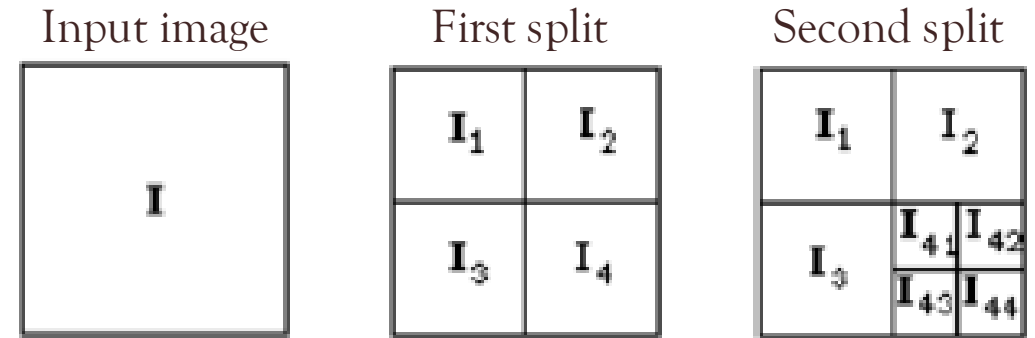
CLUSTERING: REGION GROWING

- Bottom-up method where a set of small areas are iteratively **merged** according to similarity constraints:
 1. Arbitrary **seed** pixel selected and compared to the **neighboring pixels**
 2. **Add similar neighbors**, increase the region's size
 3. When the region growth stops, continue with another seed pixel
 4. Continue until all pixels belongs to a region
- Current region dominates the growth process
- Different seeds (may) → different results
- Problems can occur if the seed point lies on an edge



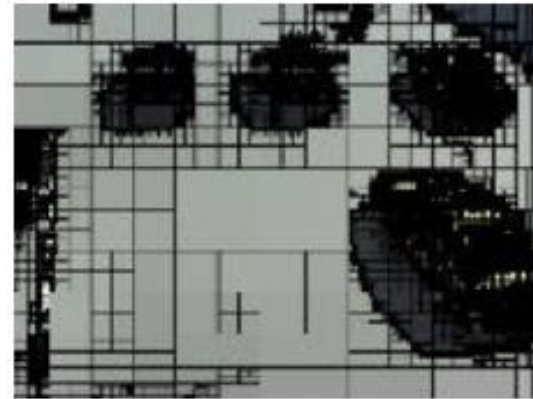
CLUSTERING: REGION SPLITTING

- Top-down method where the image is **broken** into a set of disjoint regions:
 1. The whole image is the area of interest
 2. Check if all pixels contained in the region **fulfill** some **similarity constraint**
 - If True, the area of interest is a **region** in the image
 - If **False**, **split** the area of interest and consider each sub-area as a new area of interest
 3. Continue until no further splitting occurs.
Limit (worse) case $\rightarrow$ areas = pixels



CLUSTERING: REGION SPLITTING

- Example:



- Since the split affects the whole area of interest, the final segmentation may contain identical or very **similar neighboring regions**
- A **merging** step after each split: compare adjacent regions and merge them if necessary

Merge

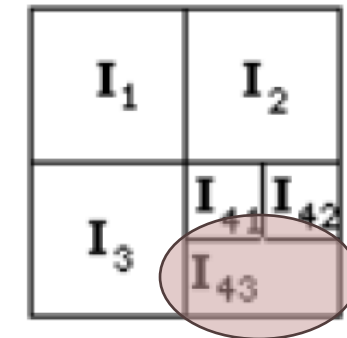


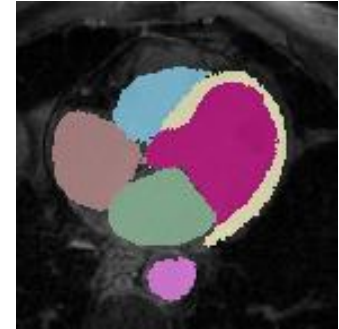
IMAGE SEGMENTATION

Many different approaches:

1. **Intensity**-based:
 - thresholding (histogram-based)
 - clustering
 2. Region growing
 3. **Contour** based, etc.
- 
- “Classical”
-
4. Multi-atlas segmentation (medical imaging)
 5. Deep Learning: U-net (+ 3D-Unet, V-net), nn-Unet, etc.
 6. Foundation models
- 
- ML / AI

MULTI-ATLAS SEGMENTATION

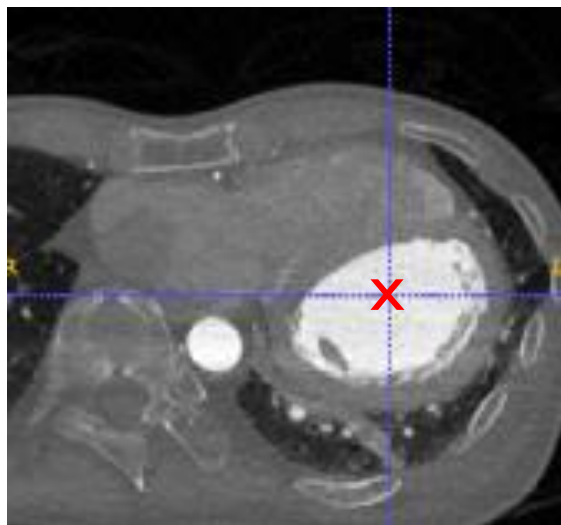
- Automatic segmentation method mainly used in medical image analysis
- An **atlas** is a reference image (already segmented / labelled)
- The atlas is used as a **guide** to segment other images
- To reduce the dependence on the atlas, **several different ones** are typically used
- Steps:
 - **Registration** (alignment): each atlas is registered with the image to be segmented. Linear or non-linear transformations are applied to make the structures match
 - **Label propagation and fusion**: the labels from each atlas are transferred to the image to be segmented and fused to obtain a final segmentation



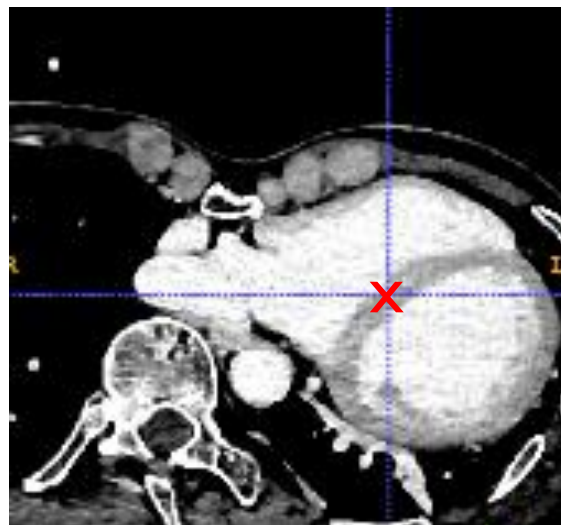
Atlas example:
MRI + anatomical
labels

IMAGE REGISTRATION

arterial-CT



LIE-CT



arterial-CT registered
to LIE-CT

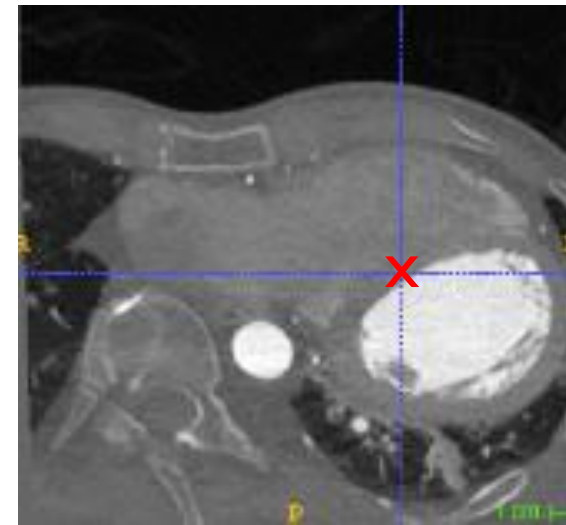
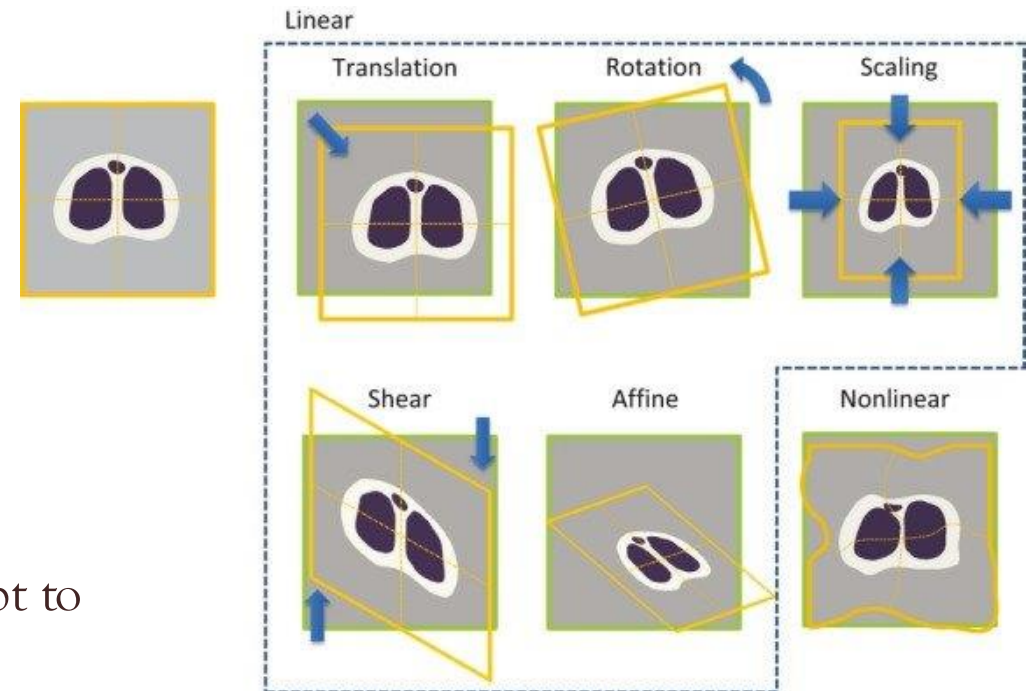


IMAGE REGISTRATION

Types of registration:

- Intensity-based: compare intensity patterns in the images using correlation metrics
- Feature-based: search for correspondences between features in the images such as keypoints, lines, contours...
- According to the type of transformation:
 - **Rigid**: rotation, translation and scaling
 - **Affine**: rigid + shearing
 - **Non-rigid** or **elastic**: capable of locally deforming the source image to align with the target image

Unable to adapt to
local changes

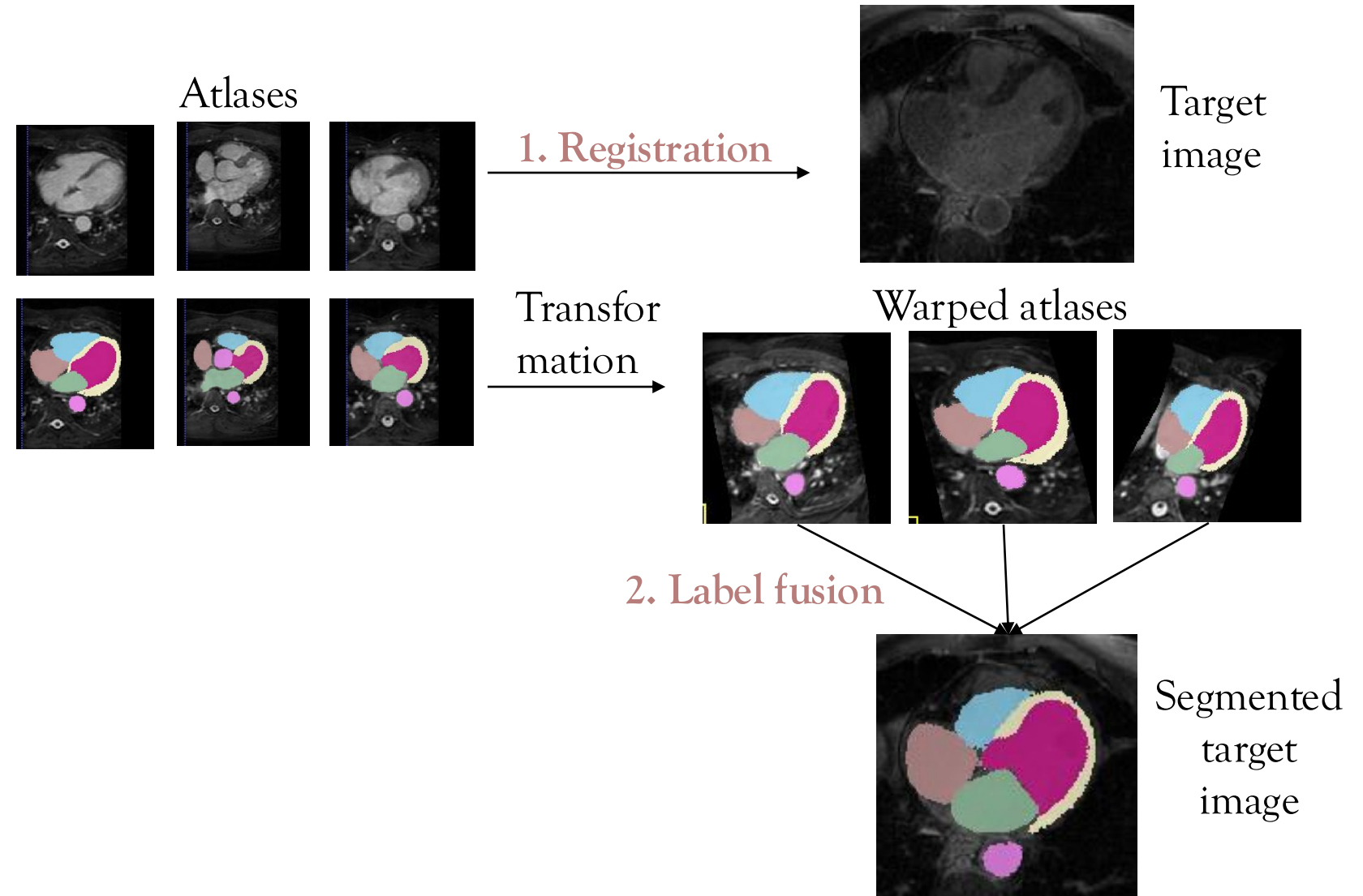


<https://pmc.ncbi.nlm.nih.gov/articles/PMC3746120/>

MULTI-ATLAS SEGMENTATION

- Two main steps:

1. **Registration** of several atlases to the target image
2. **Label fusion**



LABEL FUSION

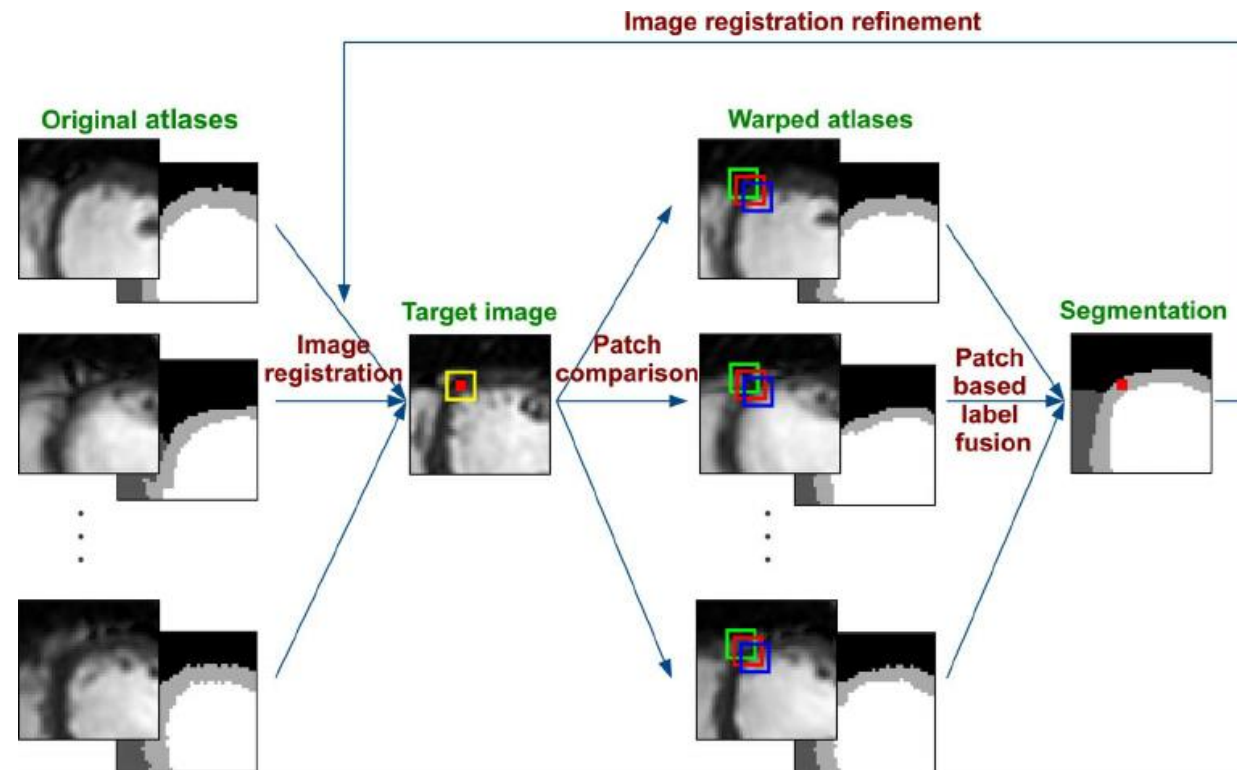
Types of label fusion:

- Majority voting
- Weighted methods according to the similarity between images (the most similar images have greater weight)
- STAPLE (Simultaneous Truth and Performance Level Estimation, Warwick et al. 2004). Iterative method:
 - Preliminary segmentation using majority voting
 - Estimation of the precision of each atlas used
 - In the next iteration, atlases with **lower segmentation precision** will have a lower weight in the calculation of the fused segmentation
 - Repeat until it converges (the fused segmentation no longer changes)

LABEL FUSION

Types of label fusion:

- Patch-based: use the better atlases **locally**



<https://pubmed.ncbi.nlm.nih.gov/23568495/>

MULTI-ATLAS SEGMENTATION

- Robust to anatomical variability
- Able to use anatomical information from **multiple real examples**
- **Computationally expensive**, mainly due to the multiple registrations (especially elastic)
- The result highly depends on the quality of the atlases: they must be similar to the target image and be well-segmented
- **Atlas selection:** use only the most suitable atlases
- Usually done after the registration stage (computationally expensive if there are many atlases) → only the atlases that best aligned with the target image are selected and used
- There are more efficient alternatives to select them before registration, using similarity metrics for example

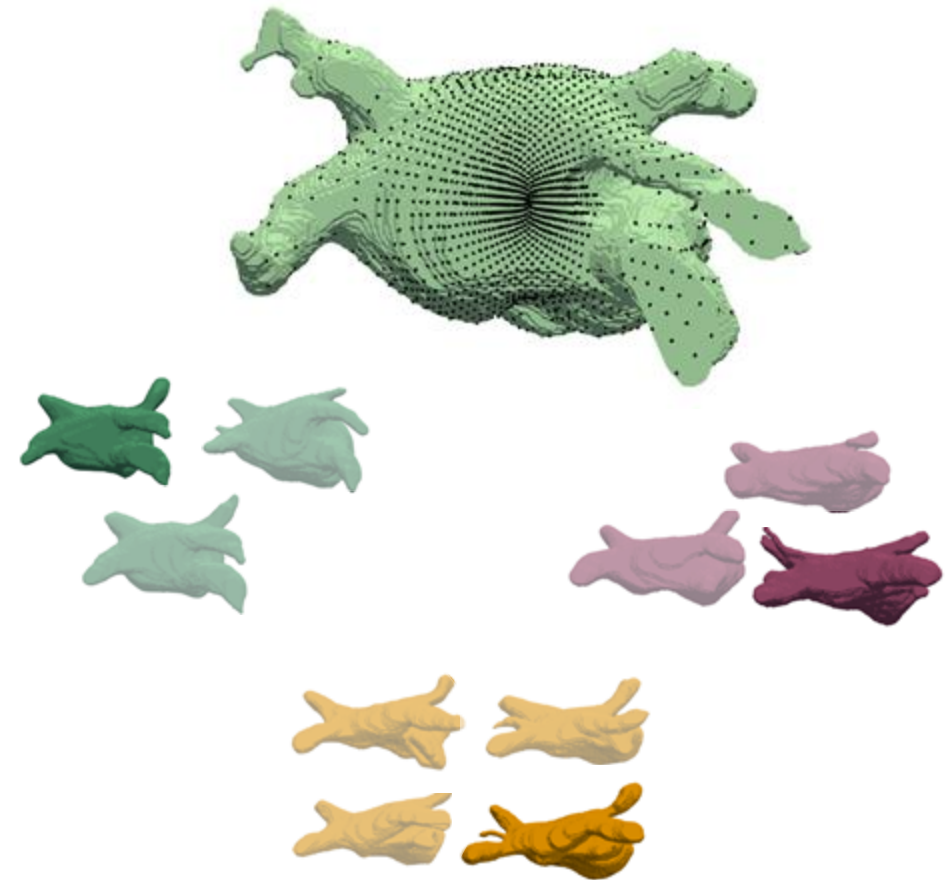
MULTI-ATLAS SEGMENTATION: EXAMPLE

- Atlas selection based on shape:
 - Point Distribution Model (PDM)
 - Principal Component Analysis (PCA)
 - Unsupervised clustering
- Identification of the type of left atrial morphology (common left trunk, extra pulmonary vein, etc.)
- Selection of the most similar atlases **prior to registration** and multi-atlas segmentation using only those atlases

[Home](#) > [Statistical Atlases and Computational Models of the Heart. Atrial Segmenta...](#) > [Conference paper](#)

Left Atrial Segmentation Combining Multi-atlas Whole Heart Labeling and Shape-Based Atlas Selection

[Marta Nuñez-García](#) ✉, [Xiahai Zhuang](#), [Gerard Sanroma](#), [Lei Li](#), [Lingchao Xu](#), [Constantine Butakoff](#) & [Oscar Camara](#)

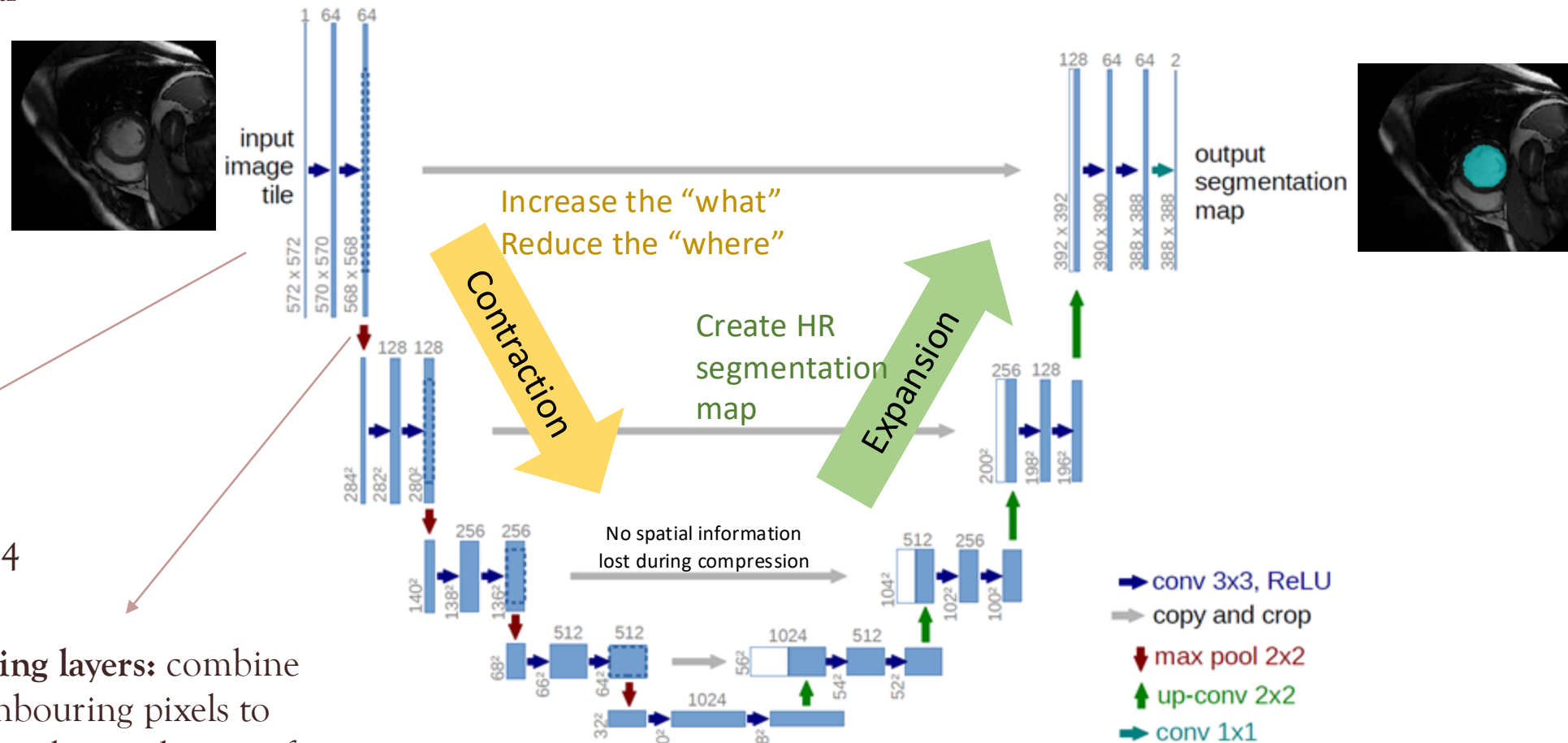


DEEP-LEARNING BASED SEGMENTATION: U-NET

- **U-net:** Convolutional Neural Network
- Extensively used for segmentation
- Extension to 3D

Convolution layers: apply multiple convolution filters, extracting features from the input. In this case: 64

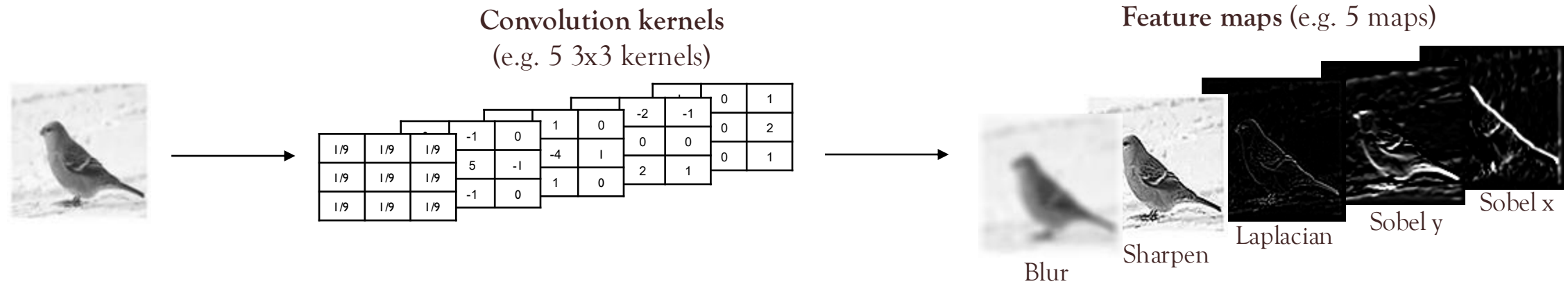
Pooling layers: combine neighbouring pixels to reduce the resolution of the feature maps



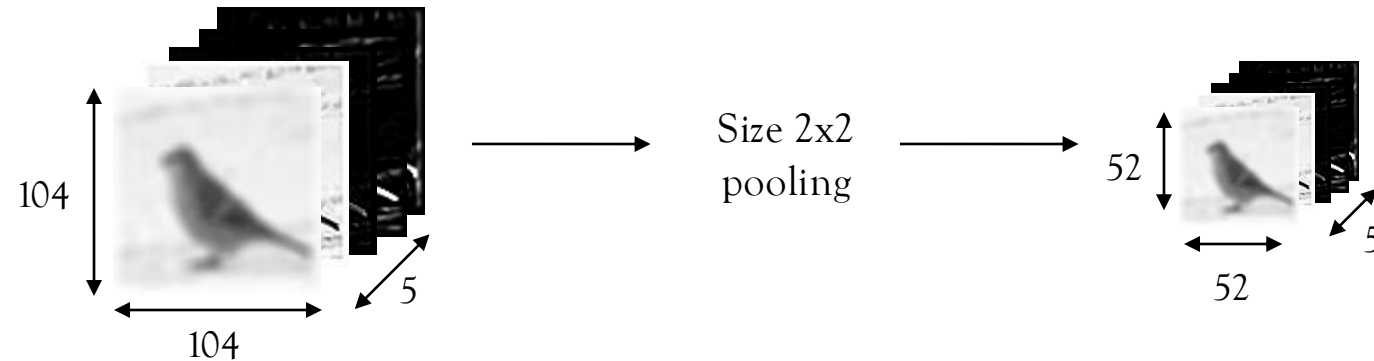
Ronneberger et al. MICCAI 2015

BUILDING BLOCKS OF A CONVOLUTIONAL NEURAL NETWORK (CNN)

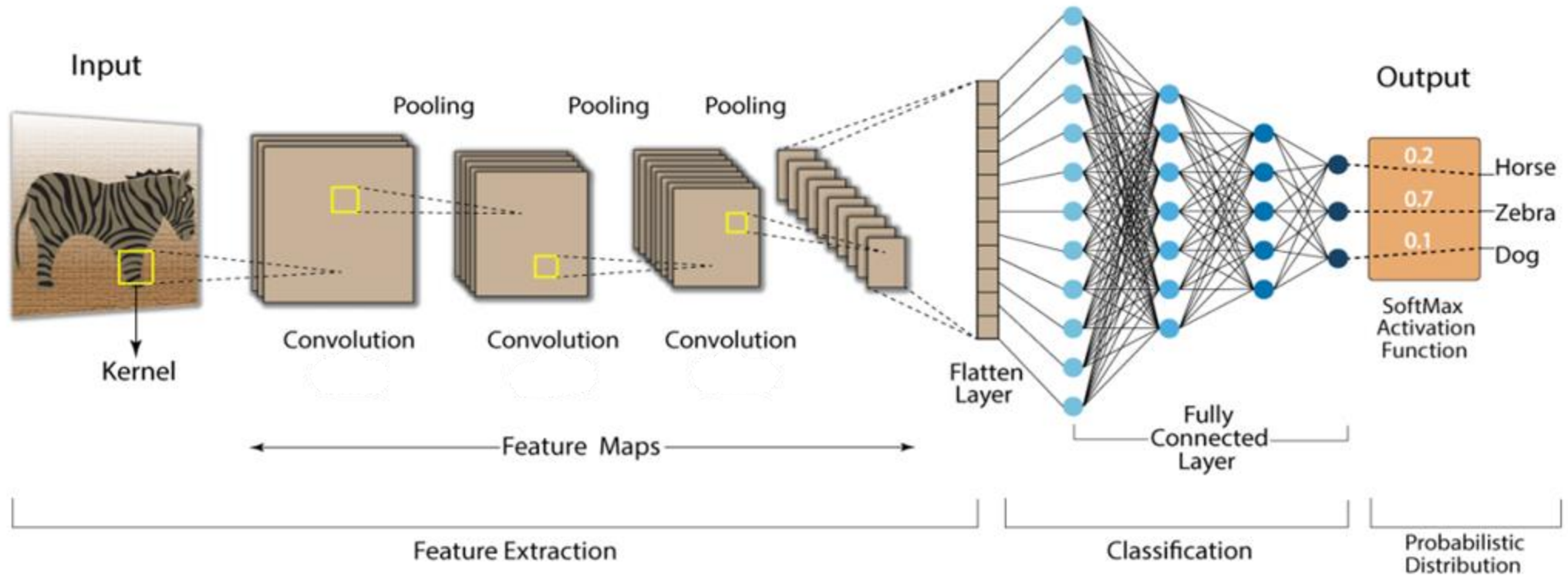
- **Convolution layers:** apply different convolution filters and compute different feature maps



- **Pooling:** combine neighboring pixels to reduce the resolution of the feature maps



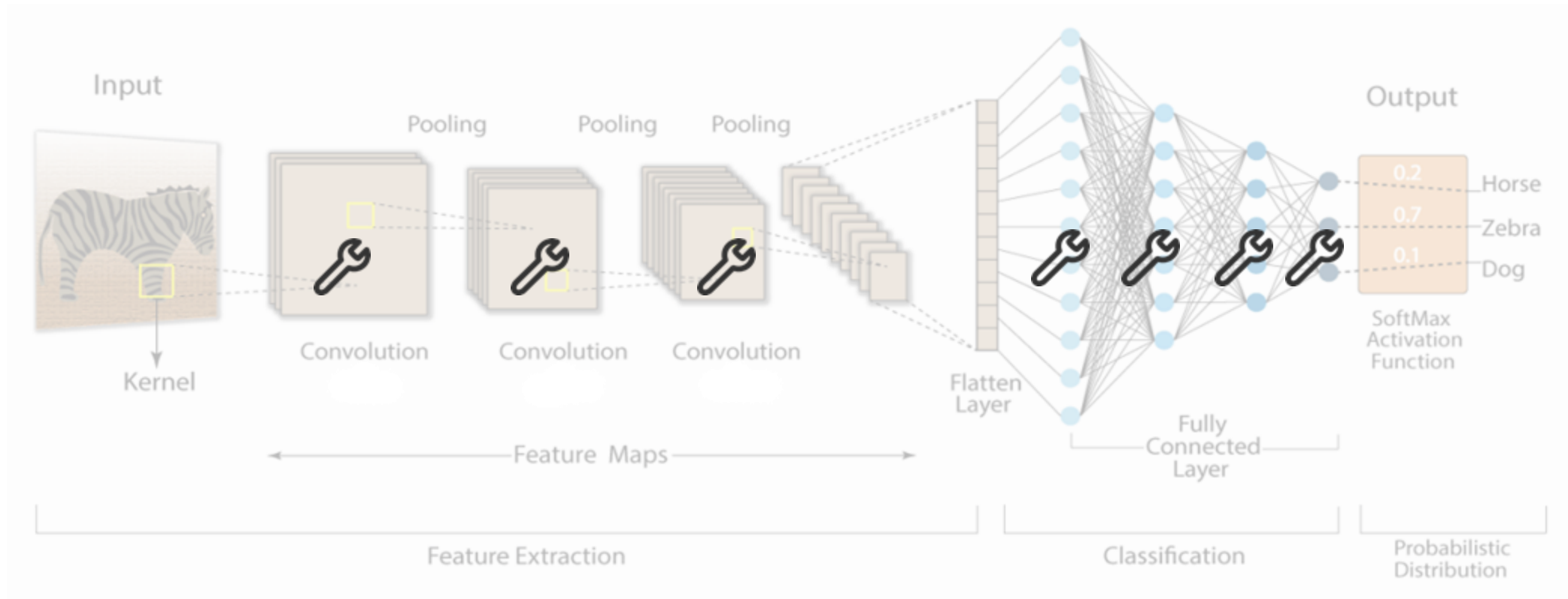
EXAMPLE OF CNN FOR CLASSIFICATION



analyticsvidhya.com

EXAMPLE OF CNN FOR CLASSIFICATION

The training process is the same as for any neural network: the values of the convolution kernels and the weights of the fully connected layers must be adjusted



analyticsvidhya.com

DEEP-LEARNING BASED SEGMENTATION: nnU-net

Article | [Published: 07 December 2020](#)

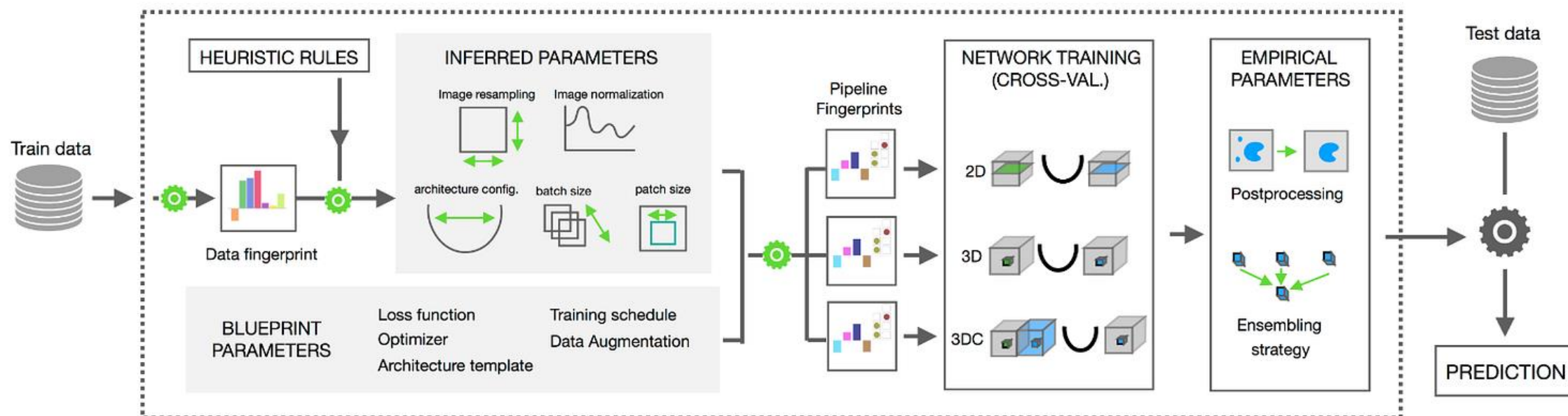
nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation

[Fabian Isensee](#), [Paul F. Jaeger](#), [Simon A. A. Kohl](#), [Jens Petersen](#) & [Klaus H. Maier-Hein](#) 

[Nature Methods](#) **18**, 203–211 (2021) | [Cite this article](#)

82k Accesses | 1503 Citations | 244 Altmetric | [Metrics](#)

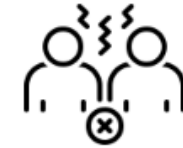
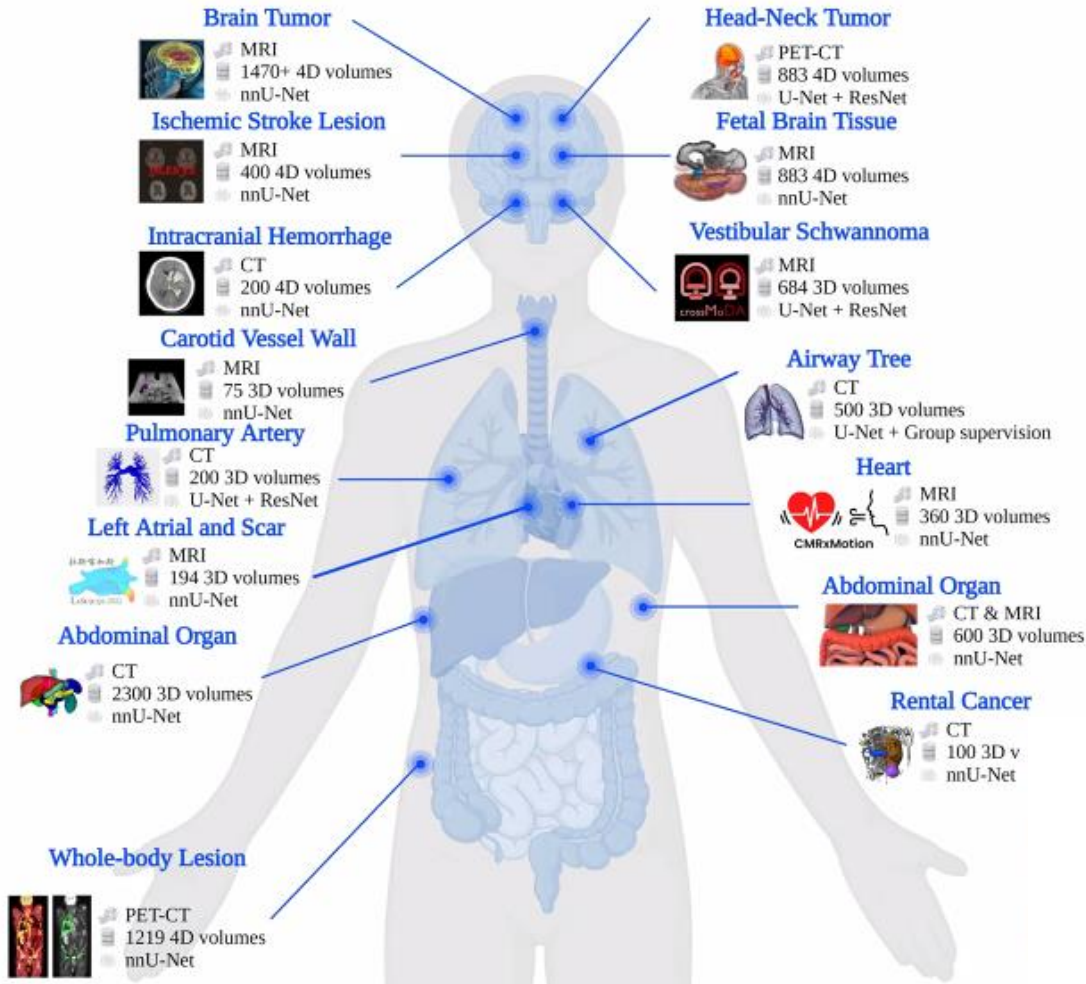
- The self-configuring (based on data properties) U-Net, nn-Unet (2021), has become a **de facto standard** for medical image segmentation



<https://medium.com/miccai-educational-initiative/nnu-net-the-no-new-unet-for-automatic-segmentation-8d655f3f6d2a>

DEEP-LEARNING BASED SEGMENTATION: nnU-net

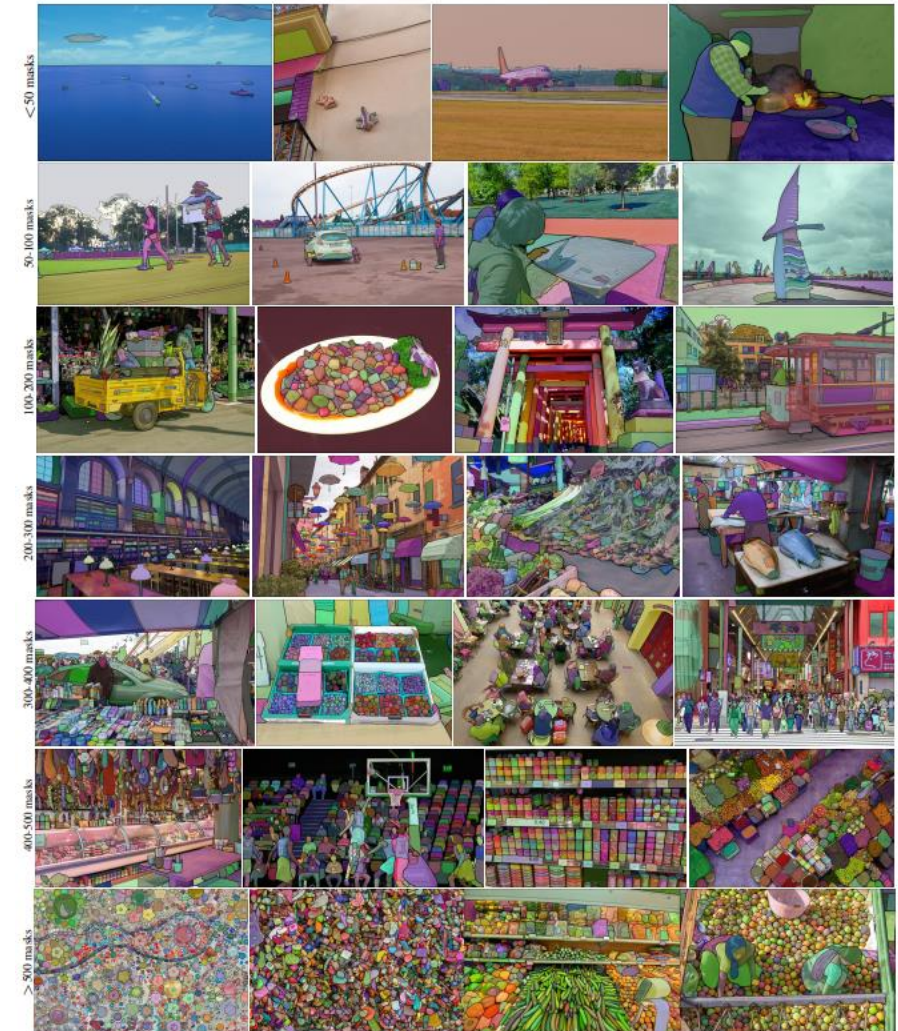
- Supervised learning → need labels!!



Expensive, boring, prone to inconsistencies (intra- and inter-observer variability; errors; etc.)

FOUNDATION MODELS

- Foundation models for general image segmentation
 - Trained with a large number of general-purpose images (natural images)
 - Learn **general representations** of visual content
 - Capable of adapting to tasks not seen during training (zero-shot learning)
-
- Segment Anything (SAM), de Meta AI Research, 2023
 - <https://segment-anything.com/>
 - Dataset
 - Demo



FOUNDATION MODELS

► Med Image Anal. 2023 Oct;89:102918. doi: 10.1016/j.media.2023.102918. Epub 2023 Aug 2.

Segment anything model for medical image analysis: An experimental study

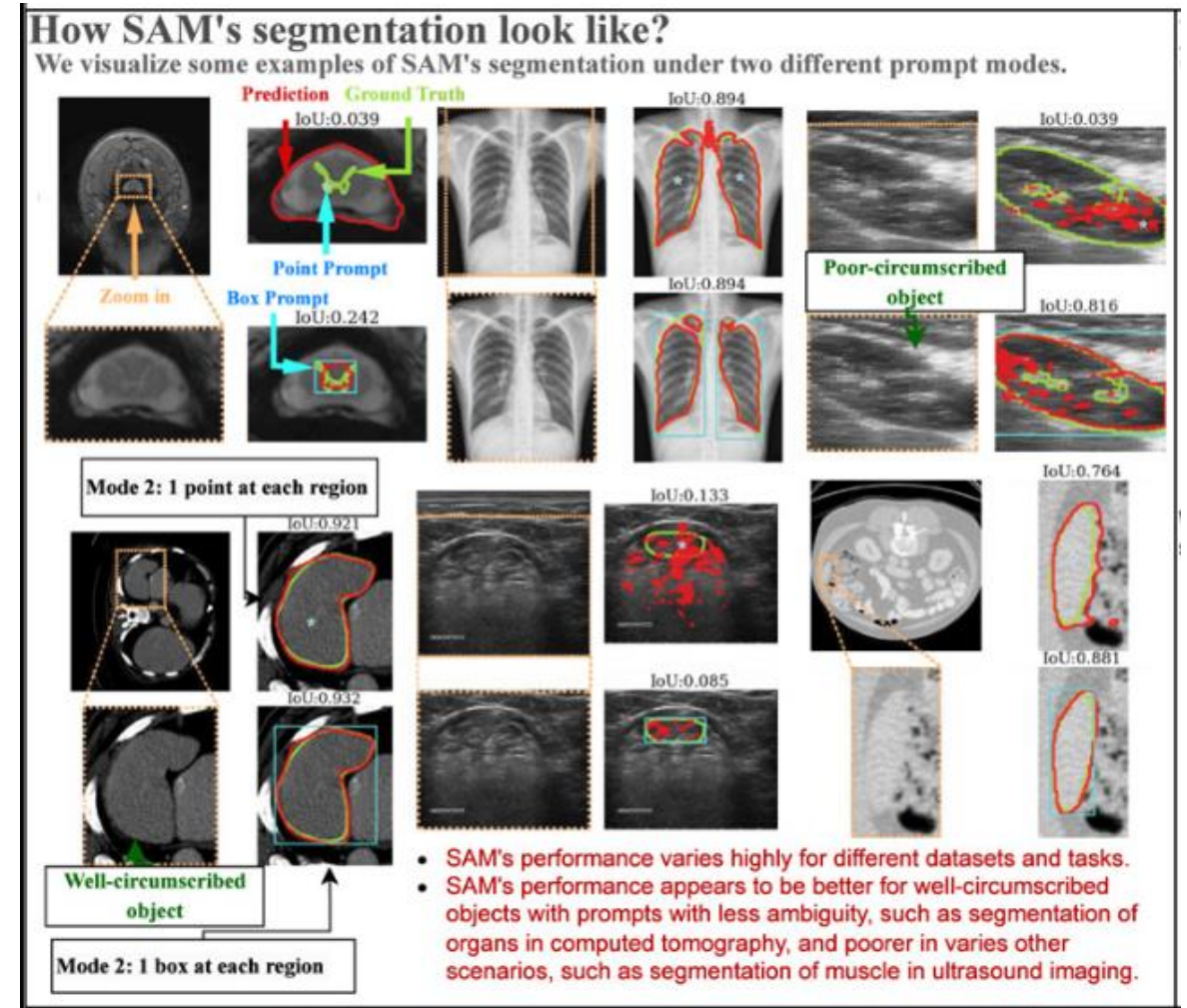
Maciej A Mazurowski ¹, Haoyu Dong ², Hanxue Gu ³, Jichen Yang ³, Nicholas Konz ³, Yixin Zhang ³

Affiliations + expand

PMID: 37595404 PMID: PMC10528428 (available on 2024-10-01)

DOI: 10.1016/j.media.2023.102918

- 11 datasets, various modalities & anatomies
- **Impressive** performance for certain medical imaging datasets
- **Moderate to poor** performance for others
- Appropriate care needs to be applied when using it



FOUNDATION MODELS

arXiv > eess > arXiv:2304.12306

Electrical Engineering and Systems Science > Image and Video Processing

[Submitted on 24 Apr 2023 (v1), last revised 17 Jul 2023 (this version, v2)]

Segment Anything in Medical Images

Jun Ma, Yuting He, Feifei Li, Lin Han, Chenyu You, Bo Wang

- Trained on large-scale **medical** datasets

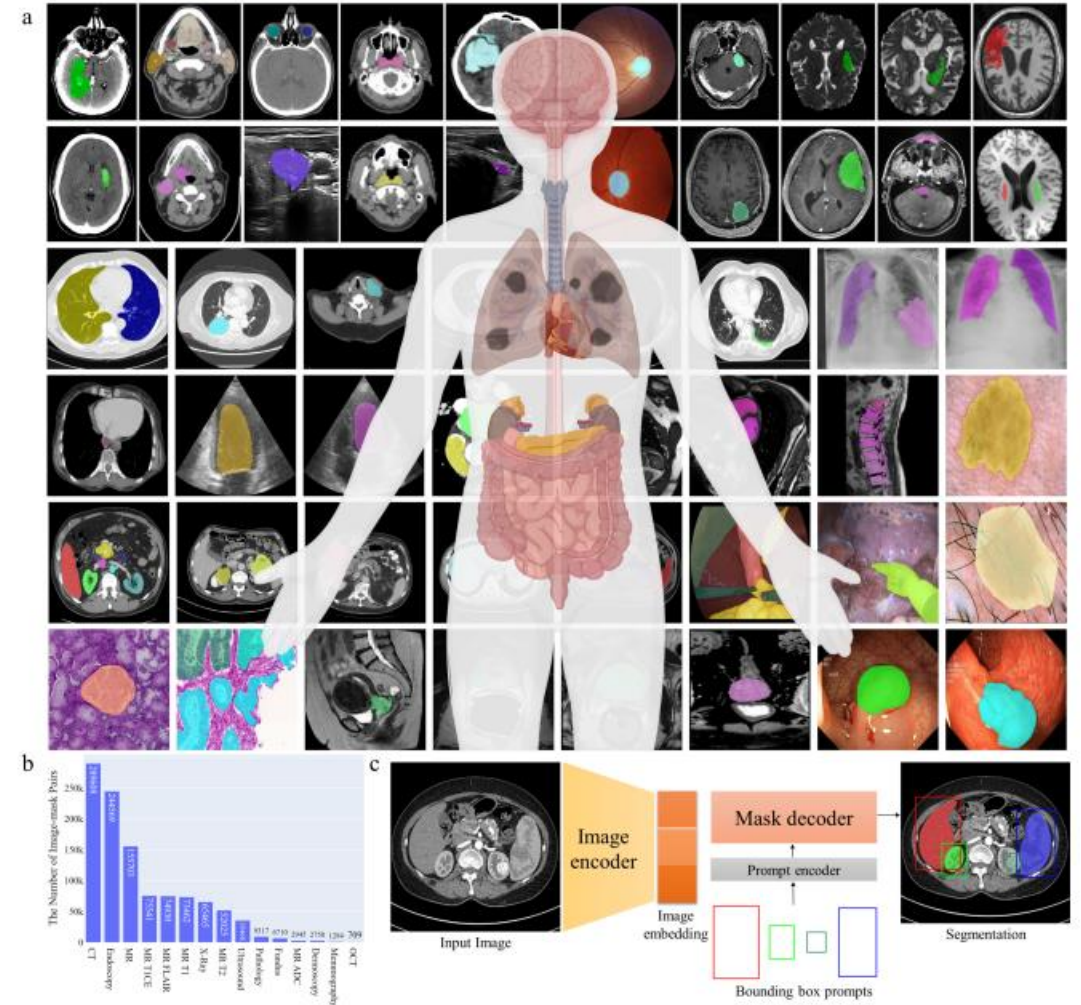


Fig. 1. **MedSAM is trained on a large-scale dataset that can handle diverse segmentation tasks.** **a**, The dataset covers a variety of anatomical structures, pathological conditions, and medical imaging modalities. **b**, The number of medical image-mask pairs in each modality. **c**, MedSAM is a promptable segmentation method where users can use a bounding box to specify the segmentation target.

FOUNDATION MODELS

arXiv > cs > arXiv:2304.06131

Computer Science > Computer Vision and Pattern Recognition

[Submitted on 12 Apr 2023]

UniverSeg: Universal Medical Image Segmentation

Victor Ion Butoi, Jose Javier Gonzalez Ortiz, Tianyu Ma, Mert R. Sabuncu, John Guttag, Adrian V. Dalca

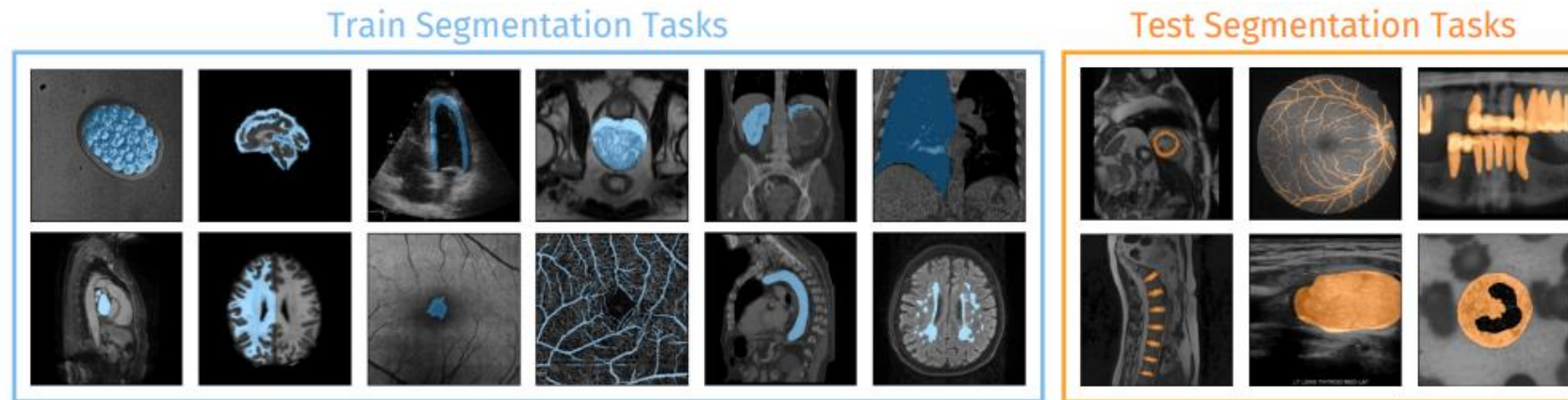


Figure 1: Medical segmentation involves many imaging types, biomedical domains, and target labels. We employ a large diverse set of training tasks (**blue**) to build a model that can segment unseen tasks (**orange**) without additional training.

- 42 multi-modal (CT, T1 and T2-weighted MRI, X-ray, ultrasound, etc.) public datasets

FOUNDATION MODELS

arXiv > cs > arXiv:2304.06131

Computer Science > Computer Vision and Pattern Recognition

[Submitted on 12 Apr 2023]

UniverSeg: Universal Medical Image Segmentation

Victor Ion Butoi, Jose Javier Gonzalez Ortiz, Tianyu Ma, Mert R. Sabuncu, John Gutttag, Adrian V. Dalca

Traditional Approach

1. Design and train a task-specific model.

Model	#Params	Runtime ms	Dice Score
PANet	14.71	240.0 $\pm$ 1.8	41.8 $\pm$ 1.3
ALPNet	43.02	527.7 $\pm$ 8.7	47.8 $\pm$ 1.1
SENet	0.92	4.1 $\pm$ 0.8	50.1 $\pm$ 1.3
UniverSeg (ours)	1.18	142.0 $\pm$ 0.4	71.8 $\pm$ 0.9
nnUNet (sup.)	17 $\times$ 1.87	17 $\times$ 1.4 $\cdot$ 10 ⁷	84.4 $\pm$ 1.0

Table 1: **Performance Summary.** For UniverSeg and each FS baseline we report model size (in millions), inference runtime, and average held-out Dice score (with bootstrapping standard deviation) . As an upper bound, we include the set of 17 individually trained task-specific nnUNets for the 6 held-out datasets, where their run-time is their cumulative required training time.

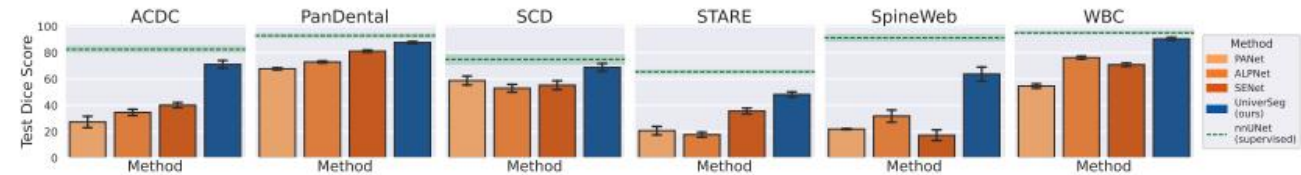
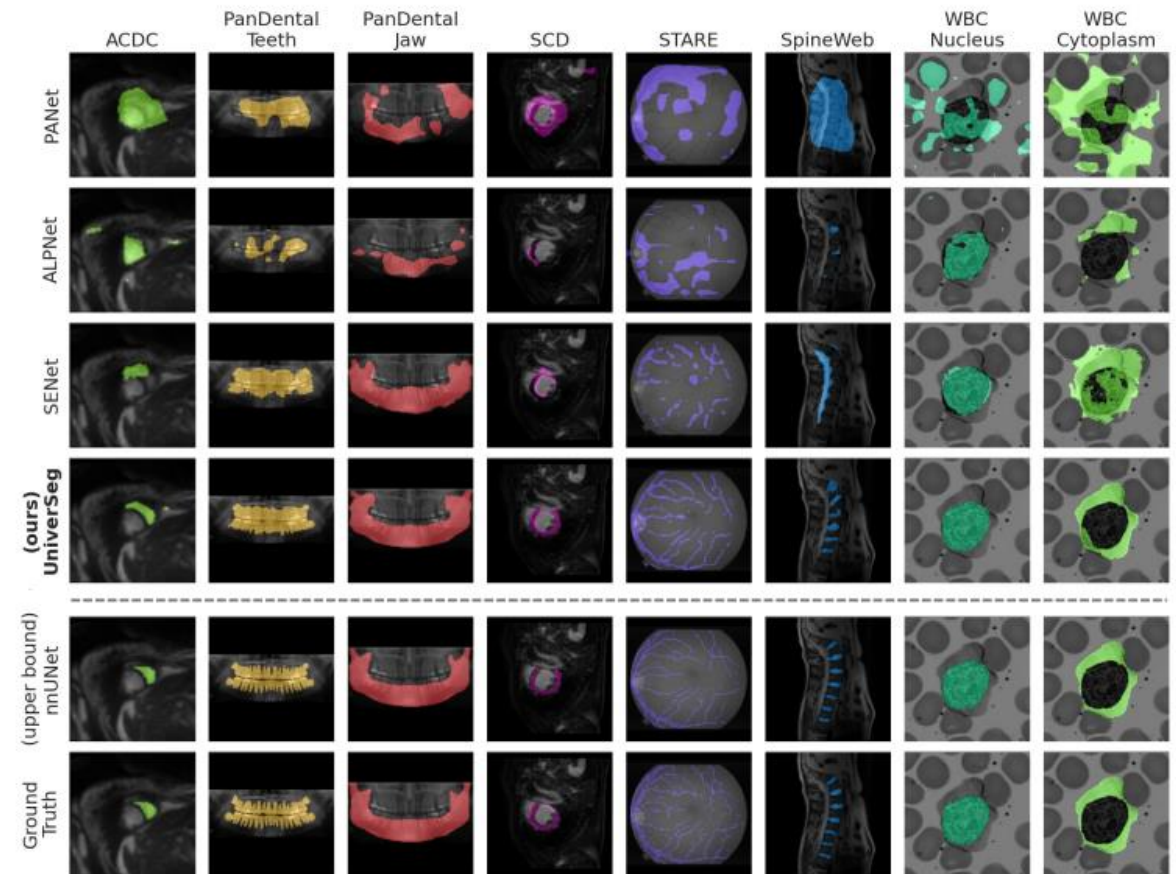
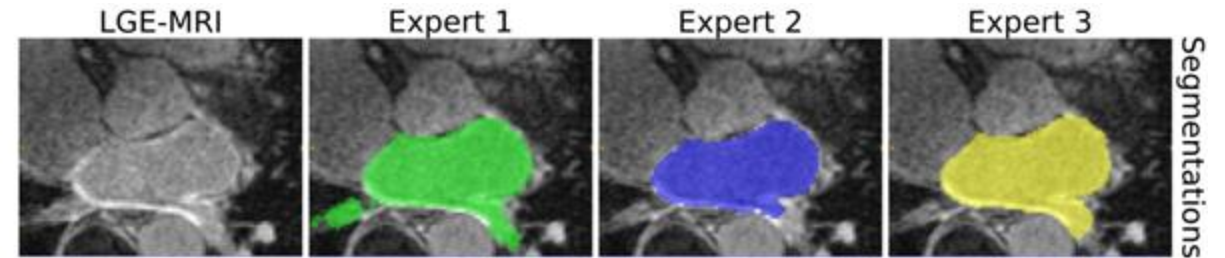


Figure 4: **Average Dice score per each held out dataset.** Performance of UniverSeg and several few-shot baselines, and the upper bound of each dataset determined by the individual fully-trained networks. For each of the unseen datasets, we average across tasks and subjects, and show the bootstrap variability in the error bars.



SEGMENTATION LABELS

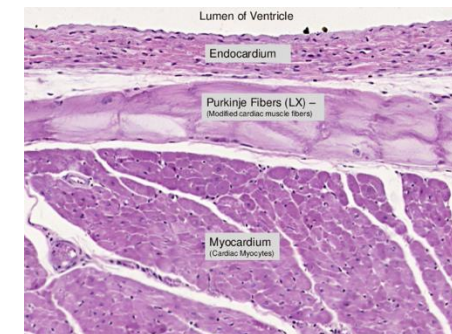
- Labels may be wrong...
- Manual segmentations (time-consuming):
 - prone to intra and inter-operator variability
 - incomplete
 - incorrect / biased, etc.



CV for medical image analysis:

Ground truth: direct observations

Gold standard: best effort to approximate the truth



<https://www.slideshare.net/jesslittl e2/cardiovascular-histology>